# PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

## COMMON INTEGRATED SOLUTION REPORT


| Team member | Student ID | Individual component |
|---|---|---|
| Priyadarshi S.A.S.D. | IT22309556 | Privacy Integration |
| Yubitha S. | IT22541284 | Dataset Generation |
| Apeksha M.K.S.R | IT22110220 | Utility Evaluation |
| Wijesinghe R.D.P.T. | IT22066916 | Risk and Compliance |


Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) Degree in Information Technology

Specialising in Cyber Security


Department of Information Technology

Sri Lanka Institute of Information Technology

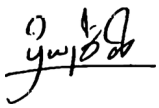
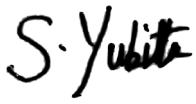
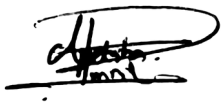
Sri Lanka


October 2026

# DECLARATION AND SUPERVISOR CERTIFICATION

**Candidate declaration.** We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

**Academic integrity and AI disclosure.** We have disclosed all use of generative AI and other automated tools, as the module policy requires. We remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this report.

| Student name | Student ID | Signature | Date |
|---|---|---|---|
| Priyadarshi S.A.S.D. | IT22309556 |  | 10/10/2026 |
| Yubitha S. | IT22541284 |  | 10/10/2026 |
| Apeksha M.K.S.R | IT22110220 |  | 10/10/2026 |
| Wijesinghe R.D.P.T. | IT22066916 |  | 10/10/2026 |

**Supervisor certification.** The above candidates carried out this research project under my supervision.

| Role and name | Signature | Date |
|---|---|---|
| Supervisor: Prof. Anuradha Jayakody | | |
| Co-supervisor: Mr. Dinith Primal | | |

# ABSTRACT

Perception models in autonomous vehicles are trained on labeled road data, and that data records real people. Driving simulators reduce the need for real footage. However, synthetic data is not private by default. The simulator is tuned with real statistics. Trained models can leak their training records. Stored files can be read or changed. On top of this, another party should be able to check any privacy claim. This report presents a framework that handles these four needs in a single pipeline. Privacy Integration applies differential privacy where real information enters. It adds Laplace noise to seven calibration statistics and trains models with DP-SGD under a hard budget limit. Dataset Generation captures CARLA frames. Each file is then hashed with SHA-256, encrypted with AES-256 and placed under role-based access control. Utility Evaluation checks the files and labels, then measures how well a detector works on real KITTI data. Risk and Compliance attacks the pipeline with separately written code and maps the evidence to the GDPR and the NIST Privacy Framework. The components share files under JSON Schema contracts, and a pipeline runner joins them in order. We tested the framework on 10,000 synthetic frames from five towns. Membership inference fell from an AUC of 0.639 on a non-private model to between 0.504 and 0.515 on every private model. This cost 0.16 to 0.21 in test AUC. All five encryption tests passed, and a single changed byte was rejected. A detector trained on synthetic data kept 10.46% of the real-data mAP50 on KITTI images, with an FID of 115.99. All six independent attacks passed. The audit also found and fixed one reporting error in the calibration budget. Overall, privacy can be applied, protected and checked across a synthetic data pipeline. The gap between synthetic and real data is still the main open problem.

**Keywords:** differential privacy, synthetic data, autonomous vehicles, privacy auditing, secure data pipeline

# ACKNOWLEDGEMENTS

We thank our supervisor, **Prof. Anuradha Jayakody**, and our co-supervisor, **Mr. Dinith Primal**, for their guidance, feedback and patience throughout this research. Their advice helped us cut a large idea down to four components that we could build and test within the project timeline.

We also thank the Sri Lanka Institute of Information Technology for the academic environment and resources that made this work possible. The academic staff of the Faculty of Computing commented on our work during the proposal and progress reviews. Their comments changed how we tested and reported it.

Each of us built a separate component, but every component depended on the others. So we thank each other for the cooperation, the code reviews and the honest checks behind many of the decisions in this report. Finally, we are grateful to the open-source communities behind the tools we used throughout this work, including CARLA, PyTorch, Opacus, Ultralytics, PyCryptodome, torchmetrics and ReportLab.

# TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT....1
1.1 Background and context....1
1.1.1 Differential privacy in a data pipeline....2
1.1.2 Membership inference and testing a privacy budget....3
1.1.3 Securing synthetic driving data....3
1.1.4 Measuring the utility of synthetic data....4
1.1.5 Independent auditing and legal requirements....5
1.2 Stakeholders and user needs....6
1.3 Alignment with the Sustainable Development Goals....7
1.4 Purpose and scope of the common report....8
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS....9
2.1 Overall problem definition....9
2.2 Root causes and overall research gap....9
2.3 Subproblem and component mapping....11
2.3.1 SP1: Privacy at the entry points....12
2.3.2 SP2: Protection of generated files....12
2.3.3 SP3: Utility and label integrity....12
2.3.4 SP4: Independent verification and compliance....13
2.4 Overall aim and group objectives....13
2.5 Scope assumptions and constraints....14
3 OVERALL SOLUTION STRUCTURE....16
3.1 Integrated solution overview....16
3.2 High level architecture....17
3.3 Component 1 summary....19
3.4 Component 2 summary....21
3.5 Component 3 summary....22
3.6 Component 4 summary....23
3.7 End to end user and data flow....25
4 COMPONENT INTEGRATION AND COHESIVE OPERATION....27
4.1 Integration strategy....27

4.2 Interfaces data contracts and shared models....28
4.3 Shared nonfunctional requirements....31
4.4 Integrated workflow and exception handling....33
4.5 Team contribution and dependency matrix....34
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION....36
5.1 Common technology stack and infrastructure....36
5.1.1 Software structure of each component....36
5.2 Development and integration process....38
5.2.1 Problems found during integration....40
5.3 Deployment and operational configuration....41
5.4 Evidence of team execution....42
6 INTEGRATED TESTING EVALUATION AND VALIDATION....45
6.1 Evaluation framework and success criteria....45
6.2 Integration and end to end testing....46
6.3 Overall results....48
6.3.1 Privacy at calibration and training....48
6.3.2 Protection of generated files....51
6.3.3 Utility of the synthetic data....53
6.3.4 Independent verification....55
6.3.5 Combined view....59
6.4 Stakeholder and user validation....60
6.5 Achievement of common objectives....61
6.6 Integrated limitations and threats....62
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION....63
7.1 Value proposition and beneficiaries....63
7.2 Market alternatives and differentiation....64
7.3 Business model and feasibility....65
7.4 Adoption intellectual property and compliance....66
7.5 Sustainable Development Goal contribution....67
8 CONCLUSION AND FUTURE DIRECTIONS....68
8.1 Overall conclusion....68

8.2 Principal integrated contribution.....69
8.2.1 Lessons learned.....69
8.3 Future directions.....70
REFERENCES.....72
APPENDICES.....76
Appendix A Integrated interface specification.....76
Appendix B Team contribution declaration.....77
Appendix C Academic integrity and AI use disclosure.....78

# LIST OF FIGURES

Figure 3.1: Overall system overview of the R26-CS-010 framework.....16
Figure 3.2: Layered architecture of the framework.....17
Figure 3.3: Internal data flow of the Privacy Integration component.....20
Figure 3.4: Secure synthetic data generation pipeline.....21
Figure 3.5: Stages of the Utility Evaluation component.....23
Figure 3.6: Structure of the Risk and Compliance component.....24
Figure 3.7: Order of hand-offs between the components.....25
Figure 4.1: Layout of an AES-256-GCM encrypted dataset file.....30
Figure 5.1: Agile SCRUM cycle followed by the group.....39
Figure 5.2: Gantt chart of the project.....43
Figure 5.3: Work breakdown structure of the project.....44
Figure 6.1: Test AUC and attack AUC by privacy configuration (mean and one standard deviation over three seeds).....49
Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks.....50
Figure 6.3: Results of the Dataset Generation security audit.....52
Figure 6.4: Class distribution and share of small objects in the synthetic dataset.....53
Figure 6.5: mAP50 and mAP50-95 of the four cross-domain experiments.....54
Figure 6.6: Per-class AP50 for training and testing within and across domains.....55
Figure 6.7: Target and final logged ε of the three private configurations.....56
Figure 6.8: Highest cosine similarity found against the re-identification threshold...58
Figure 6.9: Attack AUC reported by Privacy Integration and found by the independent replication.....59

# LIST OF TABLES

Table 1.1: Stakeholders and their needs....6
Table 1.2: Stakeholder needs mapped to framework requirements and components...7
Table 2.1: Research gaps and the components that address them....10
Table 2.2: Subproblems and the components that address them....11
Table 2.3: Group objectives, components and evidence....13
Table 3.1: Shared services, external systems and users of the architecture....18
Table 3.2: Calibration queries, sensitivities and Laplace noise scale at ε = 1.8 per query....19
Table 3.3: Training configurations of Privacy Integration....20
Table 3.4: RBAC roles and permissions....22
Table 3.5: Cross-domain evaluation protocol....23
Table 3.6: The six independent attacks of Risk and Compliance....24
Table 4.1: Integration sequence and the check at each hand-off....27
Table 4.2: Main files exchanged between components....28
Table 4.3: Privatized calibration statistics sent to Dataset Generation....29
Table 4.4: Shared nonfunctional requirements....31
Table 4.5: Shared threat model and controls....32
Table 4.6: Failure paths and responses....33
Table 4.7: Team contribution matrix....34
Table 4.8: Dependency matrix between components....34
Table 4.9: Shared outputs of the group....35
Table 5.1: Development and test environment of each component....36
Table 5.2: Main modules and scripts of each component....37
Table 5.3: Problems found during development and integration....40
Table 5.4: Stages of the deployed pipeline....42
Table 6.1: Datasets and splits used in the evaluation....45
Table 6.2: Measures used in the evaluation....46
Table 6.3: Integration and system test results....47
Table 6.4: Summary of the main results of each component....48

Table 6.5: Privacy and utility by configuration (mean ± standard deviation over three seeds).....48
Table 6.6: Paired t-tests of each private configuration against the baseline (three seeds).....49
Table 6.7: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR).....50
Table 6.8: Summary of the Dataset Generation security audit.....51
Table 6.9: Cross-domain evaluation results.....53
Table 6.10: FID interpretation scale.....55
Table 6.11: Results of the six independent attacks.....55
Table 6.12: Re-identification results by modality.....57
Table 6.13: GDPR compliance mapping with attack evidence.....58
Table 6.14: Cost of privacy at the data level and the model level.....59
Table 6.15: Comments of the second progress review panel and the team's responses .....60
Table 6.16: Achievement of the group objectives.....61
Table 7.1: Value of the framework for each customer or user group.....63
Table 7.2: Existing alternatives compared with the framework.....64
Table 7.3: Main parts of the business model.....65
Table 8.1: Future directions mapped to components and group objectives.....71
Table A.1: Integrated interface specification.....76
Table B.1: Team contribution declaration.....77

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| AES | Advanced Encryption Standard |
| AI | Artificial Intelligence |
| AP | Average Precision |
| AUC | Area Under the ROC Curve |
| AV | Autonomous Vehicle |
| CARLA | Car Learning to Act (open-source driving simulator) |
| CBC | Cipher Block Chaining |
| CPU | Central Processing Unit |
| CSV | Comma-Separated Values |
| DG | Dataset Generation (component) |
| DP | Differential Privacy |
| DP-SGD | Differentially Private Stochastic Gradient Descent |
| EQ | Evaluation Question |
| FID | Fréchet Inception Distance |
| FPR | False-Positive Rate |
| GCM | Galois/Counter Mode |
| GDPR | General Data Protection Regulation |
| GO | Group Objective |
| GPS | Global Positioning System |
| GPU | Graphics Processing Unit |
| HMAC | Hash-based Message Authentication Code |
| IEC | International Electrotechnical Commission |
| IoU | Intersection over Union |
| IP | Internet Protocol |
| ISO | International Organization for Standardization |
| IV | Initialization Vector |
| JSON | JavaScript Object Notation |
| KITTI | Karlsruhe Institute of Technology and Toyota Technological Institute (dataset) |
| LiDAR | Light Detection and Ranging |
| MAC | Message Authentication Code |

| Abbreviation | Meaning |
|---|---|
| mAP | Mean Average Precision |
| MIA | Membership Inference Attack |
| ML | Machine Learning |
| NIST | National Institute of Standards and Technology |
| PDF | Portable Document Format |
| PI | Privacy Integration (component) |
| PKCS | Public-Key Cryptography Standards |
| PNG | Portable Network Graphics |
| RBAC | Role-Based Access Control |
| RC | Risk and Compliance (component) |
| RDP | Rényi Differential Privacy |
| RGB | Red, Green, Blue |
| ROC | Receiver Operating Characteristic |
| SDG | Sustainable Development Goal |
| SGD | Stochastic Gradient Descent |
| SHA | Secure Hash Algorithm |
| SP | Subproblem |
| TPR | True-Positive Rate |
| UE | Utility Evaluation (component) |
| XOR | Exclusive OR |
| YAML | YAML Ain't Markup Language |
| YOLO | You Only Look Once (object detector) |

# 1 INTRODUCTION AND PROJECT CONTEXT

## 1.1 Background and context

Autonomous vehicles use machine learning models to detect objects, follow lanes, predict how other road users will move and decide how to drive. These models learn from large amounts of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were made by driving cars fitted with sensors on public roads. People then labeled the recordings by hand, which takes a lot of time and money.

Real road data also records people. Cameras pick up faces and license plates. GPS traces and repeated routes can show where someone lives, works or studies. Under the General Data Protection Regulation (GDPR), recordings of identifiable people count as personal data. Article 25 asks for data protection by design and by default, and Article 32 asks for suitable technical measures to protect that data [3]. So a company that trains perception models on real footage has to treat that footage as sensitive.

One common answer is to generate training data in a driving simulator such as CARLA [4], which labels every frame on its own. No real person stands in front of a simulated camera, so synthetic data looks like an easy fix for the privacy problem. However, it only solves part of the problem. The simulator has to be calibrated with real statistics, so real information still gets into the pipeline. Models trained on the data can reveal which records were in their training set [5], [6]. Stadler et al. [7] showed that synthetic data without a formal mechanism does not reliably protect its source. Generated files can be read or changed once they sit on disk [8], [9]. Finally, developers rarely give privacy evidence that someone else can check.

Our project, R26-CS-010, tackles these risks with a privacy-enhanced synthetic data generation framework for autonomous vehicle training. Four students built it. Each of us owned one component, and the components were designed to run as one pipeline. The rest of this section looks at the evidence behind each part of the problem. We follow the order in which data moves through our pipeline.

### 1.1.1 Differential privacy in a data pipeline

Differential privacy (DP) gives a formal way to limit what the output of a computation reveals about any single record [10], [11]. Dwork et al. [12] defined it as a property of a randomized mechanism. Take two datasets D and D′ that differ in one record, and any set of possible outputs S. A mechanism M satisfies ε-differential privacy if the following holds for all such D, D′ and S.

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \cdot \Pr[M(D') \in S]$$

The value ε is called the privacy budget. A smaller ε gives stronger protection, but it needs more noise. Deep learning usually uses a relaxed form called (ε, δ)-differential privacy. It allows the bound to fail with a small probability δ [11].

The Laplace mechanism is the simplest way to answer a numerical query under DP. It adds noise drawn from a Laplace distribution with scale $b = \Delta f / \varepsilon$. Here Δf is the sensitivity, which is the largest change that one record can cause in the answer [12]. Two properties are important for our pipeline. First, post-processing cannot weaken the guarantee. Clipping or rescaling a private answer is therefore safe. Second, under sequential composition, k mechanisms that read the same records with budgets $\varepsilon_1$ to $\varepsilon_k$ together satisfy $(\varepsilon_1 + ... + \varepsilon_k)$-differential privacy [11]. Section 6.3 shows how this rule affected the calibration budget of our project.

For model training, Abadi et al. [13] proposed differentially private stochastic gradient descent (DP-SGD). The gradient of each training sample is clipped to a maximum L2 norm C. Gaussian noise with standard deviation σC is added to the sum of the clipped gradients. The result is then averaged over the batch of size L.

$$\tilde{g} = (1/L) \cdot ( \Sigma_i \, \mathrm{clip}(g_i, C) + N(0, \sigma^2 C^2 I) )$$

Clipping limits how far one record can move the model weights. The noise hides what is left of its effect. Mironov [14] introduced Rényi differential privacy (RDP), which gives tighter accounting over many training steps. Opacus [15] implements DP-SGD and an RDP accountant for PyTorch. It does not allow batch normalization, because that layer mixes information across the samples in a batch. Private models therefore usually use group normalization instead [16].

Some researchers have applied DP to generative models. DP-GAN [17] adds noise to the gradients of the discriminator, and DP-CGAN [18] adds label conditioning. Both were tested on MNIST and medical records, not on driving data. Tian et al. [19] used correlated differential privacy in federated learning for autonomous driving. Xie et al. [20] surveyed privacy risks in autonomous vehicles and listed methods that range from anonymization to private learning. None of these works protects the statistics that configure a driving simulator. That is the first point where real data enters our pipeline.

### 1.1.2 Membership inference and testing a privacy budget

A privacy budget is a bound on paper. To see what it means in practice, someone has to attack the model. Shokri et al. [5] introduced the shadow-model membership inference attack. The attacker trains shadow models that copy the behavior of the target model. It records how these shadow models respond to their own training records and to other records. A separate attack classifier then learns to tell members from non-members. Yeom et al. [21] linked this kind of leakage to overfitting. They showed that the loss of each sample is a strong membership signal.

Average measures can hide risk. Carlini et al. [6] showed that an attack with a modest area under the ROC curve (AUC) can still pick out a few records with high confidence. They suggested reporting the true-positive rate (TPR) at a low false-positive rate (FPR) as well. Jayaraman and Evans [22] found that the leakage measured at a given $\varepsilon$ varies a lot between settings. Nasr et al. [23] showed that a worst-case attacker can reach the formal bound of DP-SGD, while realistic attackers fall well short of it.

We took two lessons from these studies. A formal $\varepsilon$ should always be tested with a real attack. A non-private model should be attacked as well. If the attack works on the non-private model, then a failed attack on a private model can be trusted.

### 1.1.3 Securing synthetic driving data

CARLA [4] provides realistic towns, moving traffic, changing weather and several sensors, including RGB, depth and segmentation cameras. It can recreate rare or dangerous scenes without any real risk. SHIFT [24] showed that a large multi-task driving dataset can be generated entirely in CARLA. Real datasets such as nuScenes [25] and Waymo are still used as references for realism. These works focus on realism,

sensor quality and scale. They say very little about how the generated files are stored and protected.

Security surveys point to this weak spot. Gao et al. [8] named data tampering, sensor spoofing and unauthorized access as major threats to autonomous driving. Xu et al. [9] found that data leakage, insecure storage and re-identification are not handled well. Giannaros et al. [26] noted that security controls are rarely built into the data pipeline itself. For a synthetic data pipeline, three threats matter most. Tampering changes images or labels so that a trained model behaves badly. Disclosure lets someone without permission read or copy the files. Privilege misuse lets a user or a process do more than its task needs.

Standard controls exist for each of these threats. SHA-256 maps any input to a 256-bit digest [27]. Changing one bit of the input gives a completely different digest, so a stored digest works as a fingerprint of the file. The Advanced Encryption Standard (AES) is the standard symmetric block cipher, with 128-bit blocks and keys of up to 256 bits [28]. To encrypt whole files, AES needs a mode of operation. Cipher Block Chaining (CBC) gives confidentiality only, so a changed ciphertext may decrypt into corrupted data without any error. Galois/Counter Mode (GCM) adds an authentication tag that is checked during decryption [29]. With GCM, any change to the ciphertext, nonce or tag makes decryption fail. GCM has one strict rule: a nonce must never be reused with the same key. Role-based access control (RBAC) gives permissions to roles instead of to people. It supports least privilege, where each role gets only what its task needs, and deny by default.

### 1.1.4 Measuring the utility of synthetic data

Privacy is easy to reach if usefulness is ignored. A dataset of pure noise is perfectly private, but nobody can train a model on it. So a privacy-preserving pipeline has to measure how useful its output still is.

The You Only Look Once (YOLO) family treats object detection as a single regression problem over the whole image [30]. YOLOv8 is a widely used version [31]. Detection quality is usually reported as mean Average Precision (mAP), the standard measure of benchmarks such as Microsoft COCO [32]. For mAP50, a detection counts as correct if

its intersection over union (IoU) with the true box is at least 50%. The mAP50-95 score averages over IoU thresholds from 50% to 95%, so it is stricter. When some classes are much rarer than others, per-class results should be reported. An overall score can hide poor results on rare classes [33].

The Fréchet Inception Distance (FID) compares two sets of images using features from a pre-trained InceptionV3 network [34]. A lower FID means the two sets are more alike. FID does not depend on any task, so it adds a second view next to a task measure such as mAP. A fair utility study needs real data that is independent of the data used to calibrate the simulator. KITTI is a common benchmark for this purpose.

Labels are another way in for an attacker when they come from another party. Wang et al. [35] surveyed such data poisoning attacks. Shifted boxes, for example, can teach a detector to place objects in the wrong position. Injected labels can make it see objects that are not there. Labels from another component should therefore be checked before any training starts.

### 1.1.5 Independent auditing and legal requirements

Several studies show that synthetic data and trained models can still leak information. Hayes et al. [36] showed that generative models are open to membership inference. van Breugel et al. [37] proposed DOMIAS, an attack on synthetic data that targets local overfitting. Murakonda and Shokri [38] built ML Privacy Meter so that organizations can measure the privacy risk of their models.

Re-identification is a separate risk. Sweeney [39] showed that a few quasi-identifiers can make a person unique, and proposed k-anonymity as a defense. Pyrgelis et al. [40] showed that even aggregate location data can reveal whether a person is present. For vehicles, Adeboye et al. [41] showed that camera data is open to location inference, and they proposed DeepClean to reduce this risk. Parikh [42] used synthetic in-vehicle data to reduce driver profiling. Tang et al. [43] matched vehicles across cameras by comparing learned features. Their work is the reason we test whether synthetic samples can be matched to reference samples.

Metadata is another weak point. JSON, CSV and log files can reveal file paths, machine names, IP addresses or GPS points, even when the images themselves are

encrypted. The ISO/IEC 27559 standard gives a de-identification framework for both content and metadata [44].

On the legal side, Article 5 of the GDPR sets out purpose limitation, data minimization and accountability. Article 17 covers the right to erasure, Article 25 covers privacy by design and Article 32 covers security of processing. Ganev [45] discussed when synthetic data can count as anonymous. He argued that formal privacy and attack testing both matter for this question. The NIST Privacy Framework organizes privacy risk management into five functions [46]. The NIST AI Risk Management Framework lists privacy and security as parts of trustworthy AI [47]. These frameworks ask for evidence, but they do not say how to produce it for a synthetic driving pipeline. Chapter 2 builds the common problem from this gap.

### 1.2 Stakeholders and user needs

The stakeholders were first identified in the project proposal. We then refined their needs through supervisor reviews and the progress review panels. Table 1.1 lists the primary and secondary stakeholders and what each of them mainly needs.

**Table 1.1: Stakeholders and their needs**

| Stakeholder | Type | Main need |
|---|---|---|
| Autonomous vehicle developers (machine learning engineers and data infrastructure leads) | Primary | Training data at scale with an auditable privacy record, without long privacy reviews for every dataset |
| Road users (pedestrians, drivers and passengers) | Primary | Faces, plates and routes not exposed through training pipelines or trained models |
| Data protection officers and regulators | Secondary | Evidence of lawful processing that they can read and verify |
| Academic researchers | Secondary | Synthetic datasets that can be shared with a clear privacy and utility record |

A typical user is a machine learning engineer at an autonomous vehicle company. This engineer wants realistic synthetic data. They also have to show the company's data protection officer that the data, and the models trained on it, do not expose real people.

Real data carries legal risk, and current synthetic data tools come with no formal privacy proof. From these needs we took four requirements: formal privacy where real data enters, protected files, measured utility and evidence that a third party can check.

The secondary stakeholders look at the same pipeline from a different angle. A data protection officer does not run the code. They need a short, readable record that says what was protected, how it was tested and what the limits are. An academic researcher wants to share a synthetic dataset with another lab. Before doing so, they need to know that nobody can read or change the files on the way. They also want to know that the dataset can still train a model. Table 1.2 links each stakeholder need to the part of the framework that answers it.

**Table 1.2: Stakeholder needs mapped to framework requirements and components**

| Need | Framework requirement | Component |
|---|---|---|
| Real statistics must not expose road users | Formal privacy on the calibration statistics | PI |
| Trained models must not reveal their training records | DP-SGD with a hard budget limit, tested with attacks | PI |
| Stored and shared files must stay confidential and unchanged | Hashing, encryption and role-based access from capture | DG |
| Data must still train a useful detector | Utility measured against a real baseline, with checked labels | UE |
| Claims must be checkable by someone else | Signed evidence, independent attacks and plain-language reports | PI, RC |
| Evidence must link to legal duties | Mapping to GDPR articles and NIST functions | RC |

We did not interview or survey any external stakeholders. The needs above come from the proposal analysis, and the supervisors and review panels checked them.

### 1.3 Alignment with the Sustainable Development Goals

The framework mainly supports two Sustainable Development Goals (SDGs). The first is SDG 9 (Industry, Innovation and Infrastructure). Its target 9.5 calls for stronger research and technology capability. Our framework gives developers a tested way to use synthetic data with a formal privacy record. This supports privacy-preserving innovation in the automotive sector. The second is SDG 16 (Peace, Justice and Strong

Institutions). Target 16.10 covers the protection of fundamental freedoms, and privacy is one of these. The framework keeps personal data out of training pipelines and gives regulators evidence they can verify.

The link to road safety, under SDG 3 (target 3.6) and SDG 11 (target 11.2), is only indirect. Better access to safe training data might help perception models improve. However, we did not measure any effect on safety in this project. Section 7.5 discusses this contribution and its limits.

### 1.4 Purpose and scope of the common report

This report covers the common problem and the integrated solution. It explains how the four components split the problem, how they connect and what the full pipeline achieved. The four individual theses hold the literature review, design options and full experiments of each component. Here we summarize that work only where it helps to explain the complete solution.

The report has eight chapters. After this introduction, Chapter 2 defines the common problem, its root causes and the four subproblems. Chapter 3 then describes the overall solution and gives a short summary of each component. Chapter 4 explains how the components were joined, including the shared files, contracts and failure handling. The shared technology, the development process and the evidence of team execution are covered in Chapter 5. In Chapter 6, we present the integrated testing and the results and map them to the group objectives. Commercialization, ethics and sustainability are discussed in Chapter 7. The last chapter answers the common problem and lists the future work. Finally, the appendices give the interface specification, the contribution declaration and the AI use disclosure.

# 2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

## 2.1 Overall problem definition

Autonomous vehicle developers want to train perception models on synthetic data. That way they do not have to collect and store footage of real people. But synthetic data is not private by default, and right now nobody outside the team that made it can check its privacy. Real statistics go into the simulator, trained models can leak membership, and stored files can be read or changed. Privacy controls have a cost. The data may end up too weak to train a useful detector.

This gives the common problem of our project: **How can a synthetic data generation pipeline for autonomous vehicle training apply formal privacy at the points where real information enters, protect its files, measure its utility and give evidence that an independent party can verify?**

Each part of this question has a specific meaning. "Formal privacy" means a guarantee that comes with a stated budget. "Where real information enters" points to two places: the calibration statistics that tune the simulator and the training step that can memorize records. "Protect its files" covers the time after generation, when files sit on disk or move between machines. "Measure its utility" stops the pipeline from buying privacy with data that nobody can use. "An independent party" means that the person who checks a claim did not write the code that makes it. If any one of these parts is left out, the pipeline cannot be called private in a way that others can accept.

## 2.2 Root causes and overall research gap

We traced the problem to four root causes.

- **Calibration is treated as configuration.** Simulators are tuned with statistics taken from real data, and nobody protects these statistics [4]. Private generative models such as DP-GAN [17] were tested on small benchmarks, not on driving data.
- **Formal budgets are not tested.** Privacy budgets are often picked for accuracy and are rarely checked with attacks. Yet the leakage measured at a given ε can vary a lot [22], [23].

- **Files are protected late, if at all.** Synthetic datasets are usually protected after they are generated. Security controls are rarely part of the data pipeline itself [8], [9], [26].
- **Claims cannot be checked.** Utility is rarely measured against real data, and labels from another party are trusted as they are [35]. Few privacy claims are ever tested by an independent party. Regulations and frameworks ask for evidence, but they do not explain how to produce it for a synthetic driving pipeline [45], [46].

Table 2.1 lists these limitations in more detail. Each row names the part of our framework that responds to it.

**Table 2.1: Research gaps and the components that address them**

| Gap | Limitation of existing work | Approach in this project |
|---|---|---|
| Calibration privacy | Simulator statistics are computed from real data with no protection | Laplace mechanism on every statistic before it reaches the simulator (PI) |
| DP on driving data | DP training was mostly tested on benchmarks such as MNIST [13] | DP-SGD on synthetic CARLA frames with a hard budget limit (PI) |
| Formal bound and attack | Budgets are chosen for accuracy and rarely checked with attacks | Shadow-model and loss-threshold attacks on every model, with a non-private control (PI) |
| Protection at generation | Synthetic data is protected after generation, if at all | SHA-256, AES-256 and RBAC inside the capture pipeline (DG) |
| Tested controls | Security controls are rarely checked with repeatable tests | Five-test security audit that runs without the simulator (DG) |
| Measured utility | Privacy work rarely measures whether the data can still train a detector | YOLOv8 mAP on synthetic and real data, plus FID (UE) |
| Label integrity | Data from another party is trusted without checks | Structural and statistical poisoning gate before training (UE) |
| Independent verification | Privacy claims are rarely checked by an independent party | Six attack scripts written separately from the code they test (RC) |
| Legal mapping | Controls are built without a clear link to legal duties | GDPR and NIST mapping with an attack result as evidence for each article (RC) |

This leaves one overall research gap: no single framework covers the whole pipeline. Such a framework would protect real information at calibration and training, and protect the generated files. It would also measure the remaining utility and have every claim checked independently. Each of these parts depends on the others, so one component alone cannot close the gap.

### 2.3 Subproblem and component mapping

We split the common problem into four subproblems. They do not overlap, but they do depend on each other. Table 2.2 maps each subproblem to its component and owner.

**Table 2.2: Subproblems and the components that address them**

| Subproblem | Question | Component | Owner |
|---|---|---|---|
| SP1: Privacy at entry points | How can real information be protected where it enters the pipeline, and how can the protection be tested? | Privacy Integration (PI) | IT22309556 |
| SP2: Protection of generated files | How can every generated file be kept confidential, tamper-evident and limited to authorized roles? | Dataset Generation (DG) | IT22541284 |
| SP3: Utility and label integrity | Is the protected synthetic data still useful for training a detector, and are its labels untouched? | Utility Evaluation (UE) | IT22110220 |
| SP4: Independent verification | Do the privacy controls work as claimed, and how does the evidence map to legal duties? | Risk and Compliance (RC) | IT22066916 |

All four subproblems are needed. Without SP1, real information would reach the simulator and the models with no protection. Without SP2, someone could still copy or change the protected data on disk. Without SP3, the pipeline might produce private data that nobody can use, or train on poisoned labels. Without SP4, every privacy claim would depend on the word of the team that made it. The links between them run in several directions. SP2 uses the privatized statistics from SP1, and both SP3 and SP1 use the protected files from SP2. SP1 then trains on the clean labels from SP3, while SP4 checks the outputs of all three.

Each subproblem became the research question of one individual thesis. The sections below state these questions and explain why each one is needed for the common problem.

### 2.3.1 SP1: Privacy at the entry points

**Research question:** Can differential privacy applied at calibration and training stop membership inference on synthetic driving data at a measurable cost?

Real information enters our pipeline at two points. The first is calibration, where real statistics configure the simulator. The second is model training, where a model may memorize its training records. A formal ε is only a bound on paper, so it does not show on its own how much an attacker can learn. For this reason, SP1 needs both a mechanism and a test. The Laplace mechanism should privatize the seven calibration statistics before they leave the data owner. DP-SGD with an RDP accountant should keep training within a set budget. A membership inference attack should then work against a non-private model and fail against the private ones. The cost of privacy should show up as a measurable and tested drop in accuracy.

### 2.3.2 SP2: Protection of generated files

**Research question:** Can integrity, confidentiality and access control be built into a CARLA capture pipeline and shown to work with repeatable tests?

Synthetic datasets are often stored as plain files and protected later, if at all. Until then, a file can be read, copied or changed without anyone noticing. A changed training set could then harm a model in a safety-critical system. SP2 therefore asks for protection inside the capture step itself, with low overhead. Every captured file should be hashed, encrypted and handled through an access gateway. The protection also has to be shown to work. A security audit should confirm that decryption restores the exact bytes, that identical files give different ciphertexts and that a changed file is rejected.

### 2.3.3 SP3: Utility and label integrity

**Research question:** How much detection utility does privacy-preserving synthetic data keep compared with real data?

Privacy has a cost, and a dataset that cannot train a working detector has no practical use. SP3 measures that cost at the data level. It trains a YOLOv8 detector on the

synthetic data and tests it on both synthetic and real images. A second detector trained on real KITTI data gives the upper bound. FID adds a measure of how close the two image sets are. SP3 guards the training step against tampered labels as well, because the labels come from another component. Without SP3, the group could claim privacy but could not say whether the data is usable.

### 2.3.4 SP4: Independent verification and compliance

**Research question:** Can an independent auditor verify the privacy and security claims of the pipeline and link them to legal requirements?

A privacy claim made by its own developer is weak evidence. Article 5(2) of the GDPR makes the controller responsible for showing compliance, and Article 25 asks for privacy by design. Neither article says how to produce proof for a synthetic data pipeline. SP4 answers this with attacks written by a separate member who did not build the other components. Its targets are the signed reports, budget logs, calibration noise, trained models, metadata and synthetic samples. Each attack gives a clear pass or fail result. Each result is then linked to a GDPR article and a NIST function, so that a reviewer can see which duty it supports.

## 2.4 Overall aim and group objectives

The aim of the project is to design, build and evaluate a privacy-enhanced synthetic data generation framework. It should apply privacy, protect the files, measure utility and verify every claim across the whole pipeline. Table 2.3 lists the group objectives, the components involved and the evidence we used to evaluate each one. The first three objectives are the measurable targets from the project proposal.

**Table 2.3: Group objectives, components and evidence**

| No. | Group objective | Components | Evidence |
|---|---|---|---|
| GO1 | Keep the cumulative training budget within $\varepsilon = 10$ | PI, RC | Per-step budget logs and the independent budget audit |
| GO2 | Keep membership inference success at or below 55% on private models | PI, RC | Shadow-model and loss-threshold attacks and an independent replication |
| GO3 | Keep perception accuracy within 15% of a real-data model | DG, UE | Cross-domain mAP against a KITTI baseline |

| No. | Group objective | Components | Evidence |
|---|---|---|---|
| GO4 | Privatize calibration statistics before they reach the simulator, within $\varepsilon = 2$ | PI, DG, RC | Privatized file, simulator profile and reconstruction check |
| GO5 | Protect every generated file with hashing, encryption and access control | DG, UE | Security audit and manifest checks after decryption |
| GO6 | Check every label file for poisoning before training | UE, PI | Poisoning gate with a planted-defect test |
| GO7 | Verify all privacy claims independently and map them to the GDPR and NIST | RC | Six attack reports and the compliance mapping |

GO1 to GO3 come straight from the proposal and have numeric targets. GO4 to GO7 were added as the components took shape, because each subproblem needed its own measurable outcome. Every objective names at least one component and one kind of evidence. Where possible, the evidence comes from a different component than the one that did the work. GO1 and GO2, for example, are produced by Privacy Integration but checked by Risk and Compliance. We set these objectives before the final experiments. The results in Chapter 6 could then be judged against them without moving the target.

### 2.5 Scope assumptions and constraints

**Scope.** The framework covers the pipeline from aggregate calibration statistics to verified synthetic training data. This includes calibration, capture, protection, utility measurement, private training and auditing. Deploying a perception model in a vehicle is outside the scope. No generative model was trained on real records, because CARLA renders frames directly from its settings.

**Assumptions.** The calibration statistics were reference values measured from CARLA runs. They stood in for statistics from a real dataset such as the Waymo Open Dataset. We trusted the simulation host and the pipeline code. We did not fully trust storage or the people with access to it. KITTI was used only as an independent real-data baseline.

**Constraints.** We worked to a fixed academic timeline with limited hardware. Privacy training ran on a laptop GPU with 4 GB of memory, and detector training ran on a CPU. We used the KITTI dataset under its non-commercial license.

**Exclusions.** Some threats are outside the scope of the framework. They include attacks on the running simulation host, memory scraping, theft of an encryption key and side-channel attacks. The framework does not include a key management service or user authentication for the access gateway. We did not run a formal user study with external users either. Section 6.6 and Section 8.3 return to these gaps.

**Dependencies.** Each component needed outputs from another one. Dataset Generation needed the privatized statistics. Utility Evaluation and Privacy Integration needed the encrypted dataset and its manifests, and Privacy Integration also needed the clean labels. Risk and Compliance needed the signed evidence and the dataset folders. Chapter 4 explains how we managed these dependencies.

# 3 OVERALL SOLUTION STRUCTURE

## 3.1 Integrated solution overview

The framework is a single pipeline. It takes real driving statistics and turns them into synthetic training data that is protected, measured and verified. It starts on the data owner's machine, where the real aggregate statistics are computed. These statistics never leave that machine in raw form. At the other end is a release decision based on three kinds of evidence: verified privacy claims, compliance reports and utility figures. Figure 3.1 shows the flow.

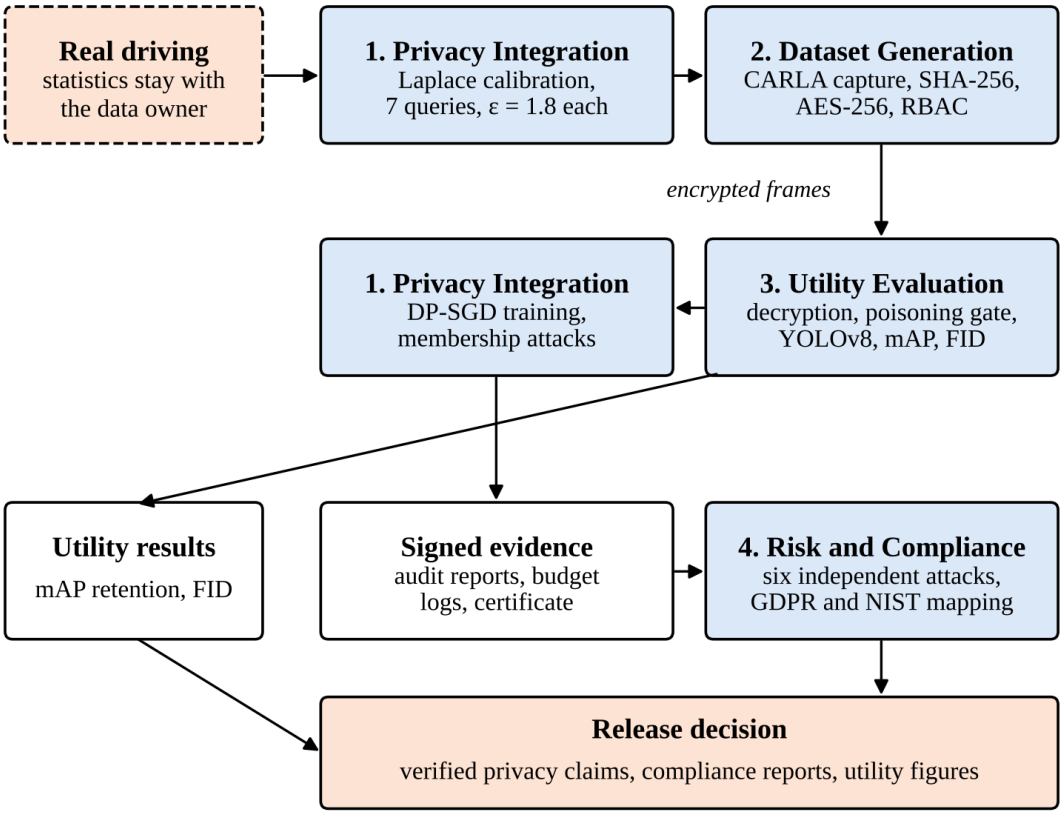



**Figure 3.1: Overall system overview of the R26-CS-010 framework**

In our framework, every privacy claim comes with a number, an attack result and a signature. Someone other than its author also checks it. A developer gets synthetic data along with a record of the work. This record shows where privacy was applied, how the files were protected, what privacy cost in utility and which independent tests passed.

Four design principles shaped the solution. We agreed on them early, and each component follows them.

- **Privacy at the source.** Real information is privatized before it reaches the simulator or a model. Nothing downstream ever sees the raw statistics.
- **Protection at capture.** Files are hashed and encrypted at the moment they are created. There is no stage where generated data sits on disk as readable plaintext.
- **Measure the cost.** Every privacy control is paired with a measurement of its cost, at the model level and at the data level.
- **Checked by someone else.** The component that makes a claim signs it. A component that did not write the code then tests it.

### 3.2 High level architecture

Another way to view the framework is as five layers, going from the real source data up to the final assurance. Figure 3.2 shows these layers.

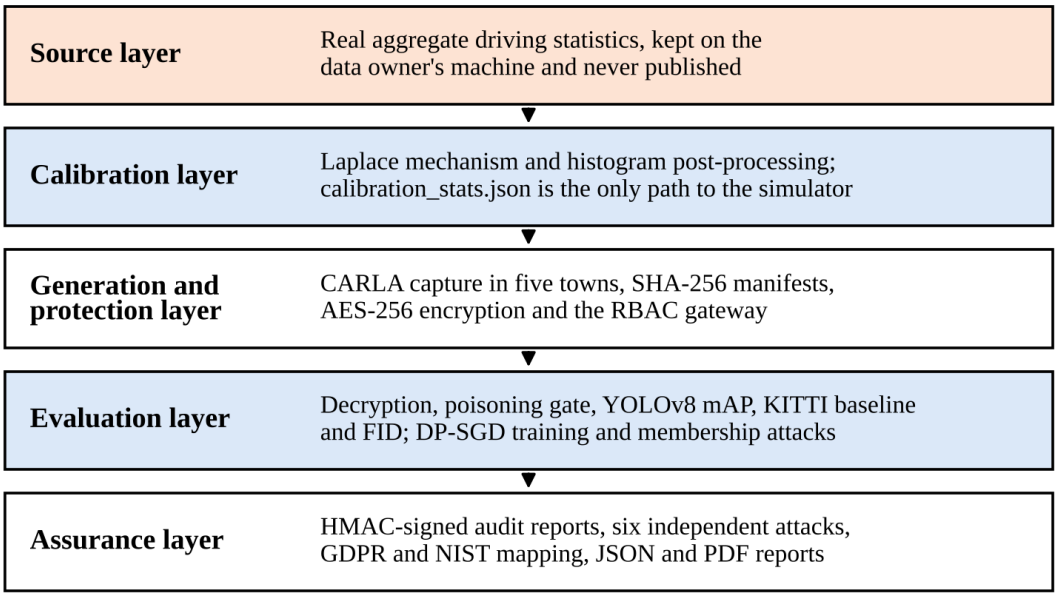



**Figure 3.2: Layered architecture of the framework**

- **Source layer.** Real aggregate driving statistics. They stay on the data owner's machine and are never published.
- **Calibration layer.** The Laplace mechanism in Privacy Integration turns the real statistics into privatized values, stored in calibration_stats.json. This file is the only path from the source data to the simulator.

- **Generation and protection layer.** Dataset Generation captures frames in CARLA and protects them with SHA-256 manifests, AES-256 encryption and an RBAC gateway.
- **Evaluation layer.** Utility Evaluation decrypts the data, checks its integrity, runs the poisoning gate and trains detectors. This layer also holds the DP-SGD training and membership inference of Privacy Integration.
- **Assurance layer.** Privacy Integration writes HMAC-signed audit reports. Risk and Compliance runs the six attacks, maps the evidence to the GDPR and NIST and writes its reports.

There are three deployment boundaries in the architecture. The data owner's machine holds the source statistics. The capture machine runs CARLA and the protection layer. The analysis machines hold the encrypted dataset and the keys needed for training, evaluation and auditing. Files only cross these boundaries when they are privatized, encrypted or signed. The people who use the architecture are the three pipeline roles of the RBAC gateway. A Simulation Engineer produces data, a Privacy Engineer checks it and an ML Researcher uses the processed datasets.

Besides the four components, the architecture has shared services, external systems and users. Table 3.1 lists them and the role each one plays.

**Table 3.1: Shared services, external systems and users of the architecture**

| Element | Type | Role in the framework |
|---|---|---|
| Shared repository | Shared service | Holds the code of all four components, the JSON Schema contracts and the tests |
| JSON Schema contracts | Shared service | Define the format of every exchanged file; every file is checked against them |
| Pipeline runner | Shared service | Runs the stages in order and stops at the first failed check |
| GitHub Actions | Shared service | Runs the unit tests of every component and an end-to-end pipeline test on each push |
| CARLA simulator | External system | Renders the synthetic towns, traffic and sensors from the privatized settings |
| KITTI dataset | External dataset | Gives an independent real-data baseline for the utility study |

| Element | Type | Role in the framework |
|---|---|---|
| Simulation Engineer, Privacy Engineer, ML Researcher | Users | The three pipeline roles of the RBAC gateway |
| Reviewers and data protection officers | Users of the evidence | Read the signed certificate and the plain-language PDF reports |

### 3.3 Component 1 summary

**Privacy Integration** (owner: Priyadarshi S.A.S.D., IT22309556).

- **Subproblem:** SP1, privacy at the points where real information enters.
- **Input:** Seven aggregate driving statistics, the encrypted dataset with its manifests and the clean labels.
- **Core method:** The Laplace mechanism privatizes each statistic at $\varepsilon = 1.8$ per query. Opacus DP-SGD with a Rényi differential privacy accountant [13], [15] trains classifiers at $\varepsilon = 1$, 3 and 8, with a hard limit of 10. A shadow-model attack [5] and a loss-threshold attack [21] then test every model against a non-private control.
- **Output:** calibration_stats.json, a private calibration audit log, HMAC-signed audit reports, per-step budget logs and a privacy certificate.
- **Contribution:** Formal privacy at calibration and training, tested with real attacks and packaged as evidence that another party can verify.

**Table 3.2: Calibration queries, sensitivities and Laplace noise scale at $\varepsilon = 1.8$ per query**

| Query | Type | Sensitivity $\Delta f$ | Noise scale b |
|---|---|---|---|
| Vehicle density per km | Scalar | 1.0 | 0.556 |
| Pedestrian density per km | Scalar | 1.0 | 0.556 |
| Average vehicle speed (km/h) | Scalar | 2.0 | 1.111 |
| Average pedestrian speed (km/h) | Scalar | 1.0 | 0.556 |
| Intersections per km | Scalar | 1.0 | 0.556 |
| Weather distribution (4 bins) | Histogram | 1.0 | 0.556 per bin |
| Time-of-day distribution (4 bins) | Histogram | 1.0 | 0.556 per bin |

Figure 3.3 shows how data moves inside the component. Calibration runs first, on the data owner's machine. Each of the seven statistics is treated as one query, and Table 3.2

lists their sensitivities and noise scales. The sensitivity is 2.0 for vehicle speed and 1.0 for the other queries. The two distributions need extra steps. Noise is added to every bin, negative bins are set to zero and the bins are scaled to sum to one. This is post-processing, so it does not weaken the guarantee.

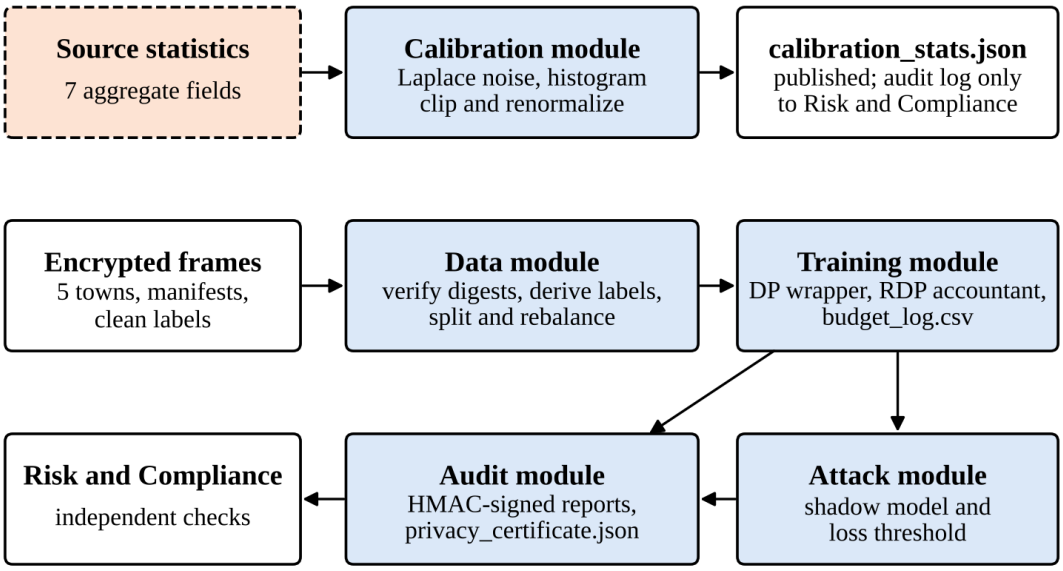



**Figure 3.3: Internal data flow of the Privacy Integration component**

**Table 3.3: Training configurations of Privacy Integration**

| Configuration | Target ε | Mechanism | Seeds |
|---|---|---|---|
| No DP (baseline) | none | Adam, 15 epochs | 42, 43, 44 |
| Config A | 1.0 | DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, $\sigma = 0.977$ | 42, 43, 44 |
| Config B | 3.0 | DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, $\sigma = 0.665$ | 42, 43, 44 |
| Config C | 8.0 | DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, $\sigma = 0.507$ | 42, 43, 44 |

The training side uses a small convolutional classifier with group normalization, since Opacus does not allow batch normalization. The task is a simple yes or no question: does a frame contain a pedestrian or cyclist that is large enough to see? Table 3.3 lists the four training configurations. Each one was trained with seeds 42, 43 and 44. The RDP accountant is read after every step, and the cumulative ε goes into a budget log. If ε ever reaches the hard limit of 10, training stops with an error.

### 3.4 Component 2 summary

**Dataset Generation** (owner: Yubitha S., IT22541284).

- **Subproblem:** SP2, protection of the generated files.
- **Input:** The privatized calibration statistics and the CARLA simulator.
- **Core method:** Captures RGB, depth and segmentation data in five towns. Each file is hashed with SHA-256 [27], checked against the RBAC policy and encrypted with AES-256 [28]. The delivered dataset used CBC mode. Later, the engine moved to GCM, which adds tamper detection [29].
- **Output:** Encrypted town folders with SHA-256 manifests, and a security audit report.
- **Contribution:** Confidentiality, integrity and least-privilege access, starting from the moment of capture.

Figure 3.4 shows the order of the protection steps. We chose this order on purpose. The digest is taken from the original plaintext as early as possible. The access check comes before any operation on the data, and encryption is the last step before data leaves the component. The security code lives in two classes. SecurityEngine does the hashing and encryption, and RBACGateway checks permissions. Neither class depends on CARLA, so both can be tested without the simulator.

The access gateway follows the real division of work in our group. Table 3.4 lists the three roles and what each one may do. A request is granted only if the role exists and the requested permission is in its list. Every other request is denied, and each decision is printed so that it can be reviewed.

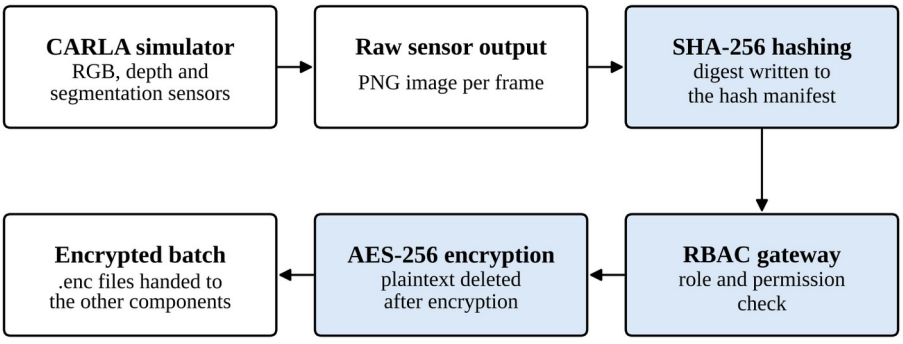



**Figure 3.4: Secure synthetic data generation pipeline**

**Table 3.4: RBAC roles and permissions**

| Role | Permitted operations | Purpose |
|---|---|---|
| Simulation_Engineer | create, verify, encrypt | Produces and protects raw simulation output |
| Privacy_Engineer | verify | Checks integrity before privacy processing |
| ML_Researcher | read_processed | Uses processed datasets only |

### 3.5 Component 3 summary

**Utility Evaluation** (owner: Apeksha M.K.S.R, IT22110220).

- **Subproblem:** SP3, utility and label integrity.
- **Input:** The encrypted dataset, its manifests and labels, and the KITTI dataset as an independent real baseline.
- **Core method:** Decrypts the data with manifest checks and runs a structural and statistical poisoning gate. YOLOv8n detectors [31] are trained on synthetic and real data and tested across domains with mAP. FID [34] measures how close the two image sets are.
- **Output:** Clean labels, cross-domain mAP results, utility retention and the FID score.
- **Contribution:** A measured cost of privacy at the data level, plus a gate that keeps poisoned labels out of every training run.

Figure 3.5 shows the stages of the component. Decryption and the poisoning gate come first, so no detector is trained on data that failed a check. The gate has two parts. The structural check makes sure that every label line has the right fields, a valid class and coordinates between 0 and 1. Each box must have a non-zero area too. The statistical check compares the class mix of each town with the other towns. Any town that differs by more than four standard deviations is flagged.

The utility measure is built on four train and test combinations, listed in Table 3.5. Each model is tested on its own domain and on the other domain. The main figure is utility retention, which is the synthetic-to-real mAP50 divided by the real-to-real mAP50.

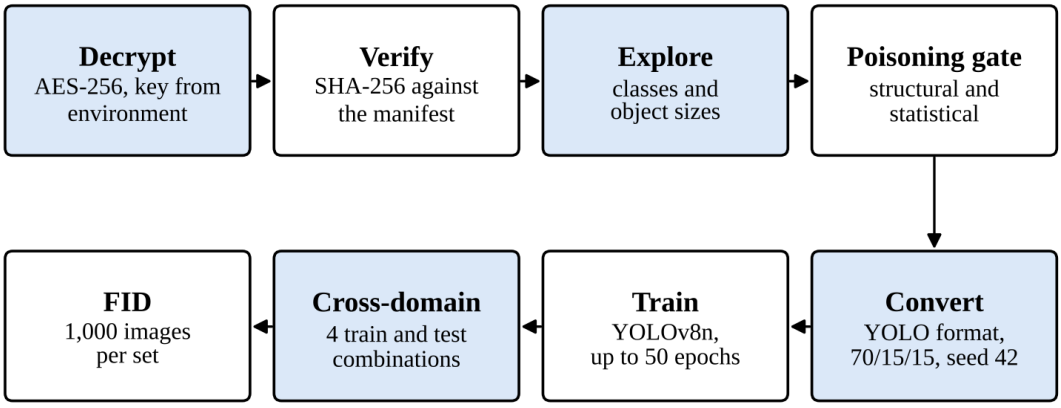



**Figure 3.5: Stages of the Utility Evaluation component**

**Table 3.5: Cross-domain evaluation protocol**

| No. | Train domain | Test domain | Purpose |
|---|---|---|---|
| 1 | Synthetic | Synthetic | Can a model learn from this data at all? |
| 2 | Synthetic | Real (KITTI) | Transfer from synthetic to real, the main utility measure |
| 3 | Real (KITTI) | Real (KITTI) | Upper bound with real data |
| 4 | Real (KITTI) | Synthetic | Reverse check on the domain gap |

### 3.6 Component 4 summary

**Risk and Compliance** (owner: Wijesinghe R.D.P.T., IT22066916).

- **Subproblem:** SP4, independent verification and legal mapping.
- **Input:** Signed reports, budget logs and the calibration audit log from Privacy Integration, the trained models and the dataset folders.
- **Core method:** Six attacks, written separately from the code they test. They cover certificate verification, a budget audit, reconstruction of the calibration values, independent membership inference, a metadata leakage scan and re-identification.
- **Output:** A JSON and a PDF report for each attack, mapped to GDPR articles and NIST Privacy Framework functions [3], [46]. A combined compliance report brings them together.
- **Contribution:** Independent evidence that the other components' controls work, with each control linked to a legal duty.

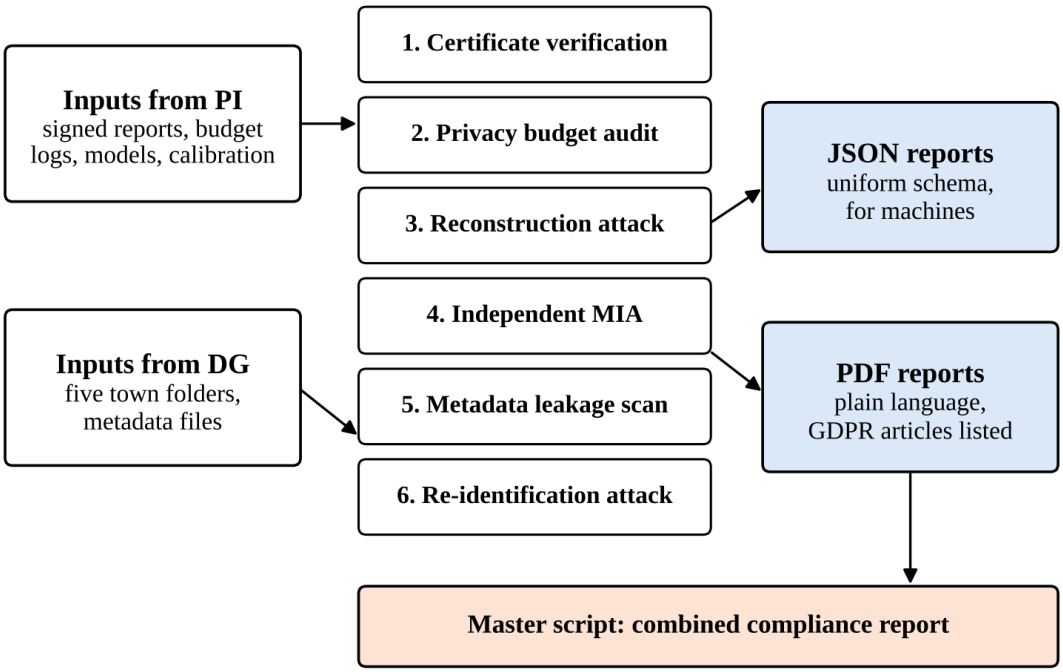



**Figure 3.6: Structure of the Risk and Compliance component**

Figure 3.6 shows the structure of the component. Table 3.6 lists the six attacks, the evidence each one reads and the rule it uses to decide pass or fail. Every script writes a JSON report for machines and a PDF report for people. The JSON reports share one schema, so a master script can run all six attacks in order and combine the results.

**Table 3.6: The six independent attacks of Risk and Compliance**

| Attack | Evidence it reads | Pass rule |
|---|---|---|
| 1. Certificate verification | Signed audit reports from Privacy Integration | Signatures and digests match; a changed copy is rejected |
| 2. Privacy budget audit | Per-step budget logs and the reported final ε | Within 0.0001 of the report, never decreasing, below 10 |
| 3. Calibration reconstruction | Published calibration values and the audit log | Ratio of observed to expected error between 0.5 and 1.5 |
| 4. Membership inference replication | Trained private and non-private models | Attack AUC below 0.55 on every private model |
| 5. Metadata leakage scan | JSON, CSV, text, YAML and log files in the town folders | No sensitive patterns found |
| 6. Re-identification | Camera, LiDAR and trajectory embeddings | Re-identification rate below 5% |

### 3.7 End to end user and data flow

A data item passes through the full solution in seven steps. Figure 3.7 shows the order of the hand-offs and which components take part at each step.

1. **Calibrate (PI).** The source statistics are privatized. Only calibration_stats.json goes to Dataset Generation. The audit log goes to Risk and Compliance and nowhere else.
2. **Generate and encrypt (DG).** CARLA is set up from the privatized values. Each captured file is hashed, access-checked and encrypted.
3. **Decrypt and verify (UE, RC).** Utility Evaluation decrypts the dataset and checks every file against its manifest. Risk and Compliance re-hashes the decrypted dataset before any training begins.
4. **Poisoning gate (UE).** Every label file is checked, and only clean labels move on.
5. **Train and attack (UE, PI).** Utility Evaluation trains YOLOv8 detectors and computes mAP and FID. Privacy Integration trains private and non-private classifiers, then attacks them.
6. **Audit and report (RC).** Risk and Compliance runs its six attacks on the signed evidence, models and dataset, and writes the compliance reports.
7. **Release decision.** The verified privacy claims, compliance reports and utility figures are reviewed together.

After calibration, the four components interact at every step. Privacy Integration and Risk and Compliance work most closely. One produces signed evidence, and the other checks it with its own separate code.

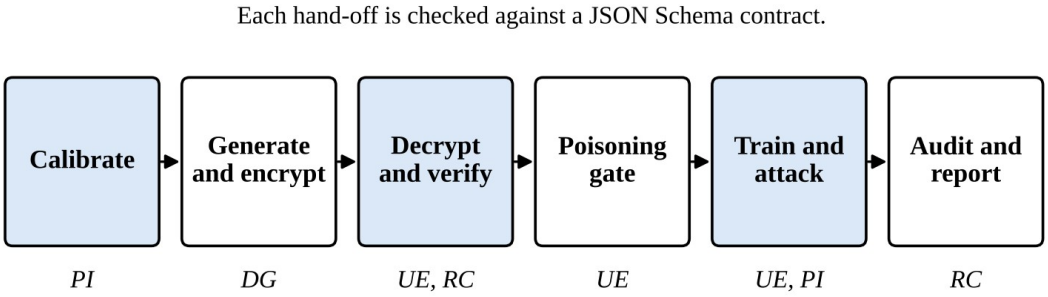



**Figure 3.7: Order of hand-offs between the components**

A single frame shows how this works in practice. The first frame kept from the Town01 capture is frame 224, because CARLA drops some frames while it warms up. Its RGB, depth, semantic segmentation and class mask images are each hashed, then encrypted into files such as rgb_f000224.png.enc. Their digests go into the Town01 manifest under the plaintext names, such as rgb_f000224.png. Utility Evaluation later decrypts these files and checks each digest against the manifest. The label file of the frame, bbox_f000224.txt, passes through the poisoning gate. Only then can the frame be used to train a detector or a private classifier. At the end, Risk and Compliance scans the metadata of the same town folder and re-hashes the decrypted files. By the time any result based on this frame is reported, three components have checked it.

# 4 COMPONENT INTEGRATION AND COHESIVE OPERATION

## 4.1 Integration strategy

We used a file-based integration strategy and agreed on the file contracts before building. Each member built and tested one component alone, but all components shared one repository and one set of data contracts. Components never called each other directly. Instead, they passed files to each other, and every file was checked against a JSON Schema contract. So each component could be built and tested on its own machine, then joined without changing its internal code.

We integrated the components in the order the data flows. The calibration hand-off between Privacy Integration and Dataset Generation came first, because no capture could start without it. Next, the encrypted dataset went to the other components. The clean labels from Utility Evaluation followed, and the signed evidence for Risk and Compliance came last. A pipeline runner joins the components in this order, and an end-to-end pipeline test runs on every push.

Each interface had one owner, the producer of the file. The producer defined the contract, and the consumer checked every file against it. This handled three integration risks. First, a schema error stopped the pipeline at the stage where it happened, so it could not cause a silent error later. Second, each key traveled on its own path, so no single component held every secret. Third, Risk and Compliance checked the others independently, so an error in one component could be caught by another.

**Table 4.1: Integration sequence and the check at each hand-off**

| Step | Hand-off | Contract owner | Check at the hand-off |
|---|---|---|---|
| 1 | Privatized calibration statistics, PI to DG | PI | Schema check; the simulator profile is compared with the privatized file |
| 2 | Encrypted towns and manifests, DG to UE, PI and RC | DG | Every decrypted file must match its SHA-256 manifest entry |
| 3 | Clean labels, UE to PI | UE | The poisoning gate must give a PASS verdict |
| 4 | Signed evidence and budget logs, PI to RC | PI | HMAC signature and SHA-256 digest must match |

| Step | Hand-off | Contract owner | Check at the hand-off |
|---|---|---|---|
| 5 | Attack summary, PI to UE | PI | Schema check before the joint privacy and utility figures |
| 6 | Compliance reports, RC to the release folder | RC | Shared JSON schema, combined by the master script |

Integration happened in three kinds of environment. Each member first ran their component on their own machine, listed later in Table 5.1. The shared repository then brought the code together. On every push, continuous integration ran all unit tests and the end-to-end pipeline test. Finally, the full experiment ran on the real five-town dataset, with each component working on the files handed over by the one before it. Table 4.1 shows the integration steps in the order we completed them, with the check that guarded each hand-off.

### 4.2 Interfaces data contracts and shared models

Table 4.2 lists the main files passed between components. Appendix A gives the format and validation rule for each one.

**Table 4.2: Main files exchanged between components**

| File | From and to | Purpose |
|---|---|---|
| calibration_stats.json | PI to DG and RC | Privatized statistics that configure CARLA |
| calibration_audit_log.json | PI to RC only | Noise per query for the reconstruction and composition checks; never published |
| Encrypted towns and manifests | DG to UE, PI and RC | Training frames and integrity digests |
| clean_labels/ | UE to PI | Labels that passed the poisoning gate |
| audit_report.json and audit_certificate.json | PI to RC | Signed evidence for each run |
| summary.json | PI to UE | Attack AUC and test AUC for the joint figure |
| privacy_certificate.json | PI to the release folder | Combined and signed privacy verdict |
| JSON and PDF compliance reports | RC to the release folder | Evidence for each attack, mapped to GDPR and NIST |

Several models and identifiers are shared between the components.

- **Labels.** All labels use the YOLO format. Each line holds a class ID and the normalized center x, center y, width and height of one object. There are three classes: car, pedestrian and cyclist.
- **Encrypted file layout.** Each file in the delivered dataset holds a 16-byte initialization vector, followed by AES-256-CBC ciphertext with PKCS7 padding.
- **Manifests.** Each town has a SHA-256 manifest of the original plaintext files. This lets us check a decrypted file bit for bit.
- **Identifiers.** Frame numbers repeat across towns, so we added a town prefix to every file before merging the towns. Training runs are named by configuration (No DP, Config A, B and C) and seed (42, 43 and 44).
- **Keys.** The AES dataset key and the HMAC signing key are kept separate. Both are read from local environment files that are excluded from version control.
- **Metadata.** Each town folder has a CSV metadata file with one row per frame. The columns name the four encrypted image files and the label file of the frame. Other columns record the number of objects, the weather category, the time of day, the event type, the town and the security applied (SHA256+AES256).

Table 4.3 shows what calibration_stats.json contained when it was sent to Dataset Generation. These are the values that configured the simulator. The source values and the noise drawn for each query are kept only in the audit log. That log goes to Risk and Compliance and nowhere else.

**Table 4.3: Privatized calibration statistics sent to Dataset Generation**

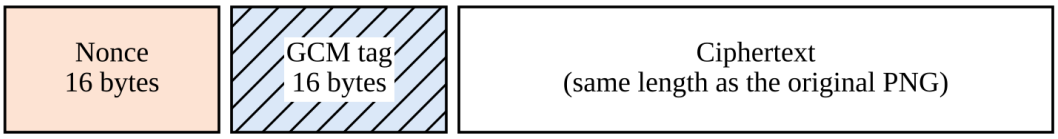
| Statistic | Privatized value |
|---|---|
| Vehicle density per km | 2.5139 |
| Pedestrian density per km | 0.9153 |
| Average vehicle speed (km/h) | 22.0135 |
| Average pedestrian speed (km/h) | 0.2789 |
| Intersections per km | 0.2796 |
| Weather: clear, cloudy, rain, fog | 0.624, 0.135, 0.241, 0.000 |

| Statistic | Privatized value |
|---|---|
| Time of day: morning, afternoon, evening, night | 0.144, 0.115, 0.575, 0.167 |
| ε per query, number of queries, total | 1.8, 7, 12.6 |

The signed evidence of Privacy Integration follows a fixed model. Each audit report records the final ε, the clipping norm C, the noise multiplier σ, δ, the limit and the number of steps. The report is signed with HMAC-SHA256, and its SHA-256 digest is stored next to it. For the non-private baseline, a signed declaration states that no privacy mechanism was applied. The budget log has one row per training step with the cumulative ε at that step.

The GCM engine, which Dataset Generation adopted after the security audit, uses a different file layout from the delivered CBC dataset. Figure 4.1 shows it. Each encrypted file stores a fresh 16-byte nonce, a 16-byte authentication tag and then the ciphertext. Every file is therefore exactly 32 bytes larger than the original. A consumer that moves to the GCM layout must read the nonce and the tag first. If the tag check fails, it must not write any output.

Each contract covers error handling as well. The consumer of a file checks it, and a failed check stops that stage. In practice, problems were reported back to the producer, who fixed them at the source. When Risk and Compliance found that the calibration total was reported wrongly, for example, Privacy Integration corrected its own code. This kept every correction in the component that owned the file.




*Layout of every .png.enc file*

**Figure 4.1: Layout of an AES-256-GCM encrypted dataset file**

### 4.3 Shared nonfunctional requirements

Table 4.4 lists the nonfunctional requirements shared by all components and how we met each one.

**Table 4.4: Shared nonfunctional requirements**

| Requirement | Target | How it was met |
|---|---|---|
| Security | No readable data or secrets outside their trusted boundary | Encryption at capture, separate keys outside version control, regression test against old keys |
| Privacy | Formal guarantee where real data enters | Laplace calibration and DP-SGD with a hard budget limit |
| Integrity | Any change to data or evidence is detected | SHA-256 manifests, GCM tags, HMAC-signed reports |
| Reliability | Errors stop the pipeline at their source | JSON Schema contracts, budget limit error, poisoning gate verdict |
| Reproducibility | Results can be repeated | Fixed seeds, fixed data splits and separate output folders per run |
| Performance | Protection does not slow the pipeline | About 15 ms to encrypt and 26 ms to decrypt a 4 KB test file |
| Maintainability | Components can be tested alone | Security and privacy code kept free of simulator code; unit tests and continuous integration |
| Usability | Results readable by non-technical reviewers | Plain-language PDF reports for each attack |

These requirements follow professional standards. SHA-256 and AES follow FIPS 180-4 and FIPS 197 [27], [28]. The evidence is mapped to the GDPR [3] and the NIST Privacy Framework [46].

The security and privacy requirements came from a shared threat model. Dataset Generation wrote it first for the stored files, and the other components adopted the same assumptions. The simulation host and the pipeline code are trusted. The storage that holds the datasets is not fully trusted, and neither are the people and processes that can reach it. The adversary can read or change files in storage, or try to use a role that lacks a permission. Table 4.5 lists the threats and the control chosen for each.

**Table 4.5: Shared threat model and controls**

| Threat | Example | Control |
|---|---|---|
| Tampering | Images in a batch are changed or replaced before training | SHA-256 digest per file with a manifest; GCM tag rejects changed ciphertext |
| Disclosure | Dataset files are copied from storage or a stolen disk | AES-256 encryption; plaintext removed after encryption |
| Privilege misuse | An ML user reads raw simulation output | RBAC gateway with deny-by-default permissions |
| Duplicate detection | Identical images give identical ciphertext | Fresh random nonce for every encryption |
| Silent corruption | A damaged file decrypts to garbage and is used | Decryption fails and no output file is written |
| Membership leakage | A trained model reveals whether a frame was in its training set | DP-SGD with a hard budget limit, tested with two attacks |
| Recovery of source statistics | Published calibration values give back the real values | Laplace noise on every query; audit log never published |

Two of the nonfunctional requirements in the template, scalability and accessibility, need a short note. For scalability, hashing reads each file in 4,096-byte blocks, so memory use stays the same whatever the file size. Encryption runs per file, and each GCM file grows by only 32 bytes. For the 6,093 images of the Town01 GCM capture, that is about 190 KB in total. Because the components exchange files, each one can run on a separate machine. For accessibility, the main users of the evidence are not programmers. For this reason, every attack writes a PDF report that explains the test, the result and its meaning in plain language. Each report ends with the GDPR articles it covers.

### 4.4 Integrated workflow and exception handling

In normal operation, the pipeline runs the seven steps of Section 3.7 in order. Each stage only reads files that the stage before it produced. Table 4.6 lists the main ways the pipeline can fail and how it responds to each.

**Table 4.6: Failure paths and responses**

| Failure | Detected by | Response |
|---|---|---|
| A file is missing a field or has the wrong type | JSON Schema contract | The run stops at that stage |
| A decrypted file does not match its manifest | SHA-256 check in UE and RC | The file is reported and held back from training |
| An encrypted file is changed (GCM engine) | GCM tag check | Decryption fails and no output file is written |
| A label file is malformed or a town's class mix shifts | Poisoning gate | FAIL verdict; training does not start |
| A user role lacks a permission | RBAC gateway | The request is denied and the decision is printed |
| The privacy budget reaches its limit | DP training wrapper | Training stops with BudgetExceededError |
| A signed report is changed | HMAC verification in RC | Verification fails and the report is rejected |

Recovery is simple, because no stage overwrites its input. Utility Evaluation never deletes the encrypted originals, so decryption can be repeated without asking for the dataset again. Each training run writes to its own folder, so a failed run can be repeated without touching the others. The one costly case is a change to the calibration statistics. That needs a new capture, because the privatized values set up the scenes themselves.

Some parts of the pipeline have a fallback. The delivered five-town dataset uses CBC, which cannot detect a changed ciphertext on its own. For this dataset, the SHA-256 manifests are the integrity check. Utility Evaluation and Privacy Integration both verify them after decryption. Utility Evaluation had no supported GPU, so its detectors were trained on a CPU. This made training slower but did not change the results, because all of its comparisons used the same hardware. Opacus secure mode could not be used with current PyTorch versions. Privacy Integration therefore trained with secure mode turned off and listed this as a known limitation in its certificate.

### 4.5 Team contribution and dependency matrix

Table 4.7 shows what each student did for the integrated solution.

**Table 4.7: Team contribution matrix**

| Student | Research tasks | Development tasks | Integration work | Test evidence |
|---|---|---|---|---|
| IT22309556 (PI) | Differential privacy, DP-SGD and membership inference | Calibration module, DP training wrapper, attacks, signed reports | Calibration hand-off, signed evidence, summary for UE, JSON contracts | 107 unit tests, attack results, budget logs |
| IT22541284 (DG) | Secure data generation and authenticated encryption | CARLA capture, SHA-256 manifests, AES engine, RBAC gateway | Encrypted dataset and manifests for all components | Five-test security audit, real folder scan |
| IT22110220 (UE) | Detection utility, domain gap and data poisoning | Decryption, poisoning gate, YOLOv8 and KITTI pipelines, FID | Clean labels for PI, manifest checks, joint figures | Cross-domain results, planted-defect test |
| IT22066916 (RC) | Privacy attacks, GDPR and NIST compliance | Six attack scripts and report generator | Verification of PI evidence and DG dataset | Six attack reports, compliance mapping |

Table 4.8 shows which component depends on which. Each row lists what that component needs from the others.

**Table 4.8: Dependency matrix between components**

| Component | From PI | From DG | From UE | From RC |
|---|---|---|---|---|
| PI | none | Encrypted dataset | Clean labels | Composition check |
| DG | Privatized statistics | none | none | none |
| UE | Attack summary | Encrypted dataset | none | none |
| RC | Signed evidence and audit log | Dataset folders | none | none |

Some outputs belong to the whole group and not to one component. Table 4.9 lists them and who worked on each. Every member reviewed these shared outputs before they were used.

**Table 4.9: Shared outputs of the group**

| Shared output | Members | What it is |
|---|---|---|
| Shared repository and JSON Schema contracts | All four | One code base and one set of file formats for the whole pipeline |
| Pipeline runner and end-to-end test | All four | Runs the stages in order and checks every hand-off on each push |
| Joint privacy and utility figures | PI, UE | Combine the attack summary of Privacy Integration with the utility results |
| Combined compliance report | RC, with evidence from PI and DG | One report built from the six attack reports |
| Research paper, website and this common report | All four | The group outputs that present the integrated solution |

We agreed that each member contributed 25% of the project. Appendix B gives the contribution declaration.

# 5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

## 5.1 Common technology stack and infrastructure

All components were written in Python and kept in one shared repository. Table 5.1 lists the software and hardware each component used.

**Table 5.1: Development and test environment of each component**

| Component | Software | Hardware |
|---|---|---|
| Privacy Integration | Python 3.11, PyTorch 2.5.1, Opacus 1.4.1 | Laptop with NVIDIA RTX 3050 Ti (4 GB), Windows 11 |
| Dataset Generation | CARLA Python API, hashlib, PyCryptodome, Jupyter | CARLA capture machine |
| Utility Evaluation | Python 3.10 and 3.11, Ultralytics YOLOv8, torchmetrics | Intel Core i5-8250U CPU |
| Risk and Compliance | Python, scikit-learn, PyTorch (CPU), Opacus 1.4.1, ReportLab | Windows 11 desktop PC |

We made the main shared choices so that the components would work together and their results could be checked. Python and PyTorch were used across the privacy, utility and audit work, so a model trained by one component could be loaded by another. PyCryptodome [48] gave the same AES implementation to the component that encrypts and to the ones that decrypt. JSON was used for every exchanged file. It can be checked against a schema, and both people and code can read it. GitHub Actions ran the tests of every component, plus an end-to-end pipeline test, on each push. We did not need a shared database or server, since the components exchange files.

### 5.1.1 Software structure of each component

Each component kept its own code in the shared repository, but all four followed the same habits. Settings are kept in one place, keys come from the environment and every run writes to its own output folder. Table 5.2 lists the main modules or scripts of each component and what they do.

**Table 5.2: Main modules and scripts of each component**

| Component | Main modules or scripts | What they do |
|---|---|---|
| PI | privacy_integration package with calibration, data, model, training, attack and audit modules; config.yaml | Privatize statistics, decrypt and split data, define the classifier, train with DP-SGD and track the budget, run the attacks, sign reports and build the certificate |
| DG | SecurityEngine and RBACGateway classes; CARLA capture wrapper; security audit notebook | Hash and encrypt files, check permissions, capture sensor data and test the security layer without CARLA |
| UE | Decryption module; poison_detection.py; prove_poison_detection.py; cross_evaluation.py; compute_fid.py | Decrypt and verify, check labels, test the gate with planted defects, run the four experiments and compute FID |
| RC | Six attack scripts; master script; ReportLab report generator | Run each attack, write JSON and PDF reports and combine them into one compliance report |

Privacy Integration was built as an installable Python package called privacy_integration. Each stage has its own command-line entry point. All settings live in config.yaml, and conflicting values are rejected when the package loads. The DPTrainingWrapper class attaches an Opacus PrivacyEngine to the model, optimizer and data loader. It calls make_private_with_epsilon to find σ for the target budget. After every step it reads the accountant and adds a row to the budget log. If the cumulative ε reaches the limit, it raises BudgetExceededError and stops the loop.

Dataset Generation kept its two security classes free of simulator code. The same classes could then run inside the capture wrapper and inside the audit notebook. Hashing reads each file in 4,096-byte blocks. For the GCM engine, a new cipher object is created for every file, which makes PyCryptodome generate a fresh random nonce. The core of the encryption method is short, as the listing below shows.

```
def encrypt_data(self, file_path):
    cipher = AES.new(self.key, AES.MODE_GCM)
    with open(file_path, "rb") as f:
        data = f.read()
    ciphertext, tag = cipher.encrypt_and_digest(data)
    enc_file_path = file_path + ".enc"
    with open(enc_file_path, "wb") as f:
        f.write(cipher.nonce + tag + ciphertext)
    os.remove(file_path)
    return enc_file_path
```

On decryption, the decrypt_and_verify method checks the tag before any plaintext is returned. If the key, nonce, tag or ciphertext is wrong, it returns False and writes no output file.

Utility Evaluation used YOLOv8n, the smallest member of the YOLOv8 family, with 3,006,233 parameters. Both detectors were trained for up to 50 epochs at 640 × 640 pixels. The batch size was 8 and the seed was 42. Early stopping used a patience of 15 epochs, and the best checkpoint was kept by validation mAP. The KITTI labels were converted to the same YOLO format. Car, Van and Truck became car. Pedestrian and Person_sitting became pedestrian, and Cyclist stayed cyclist. The Tram, Misc and DontCare classes were left out.

Risk and Compliance wrote all six attack scripts to the same pattern. Each script sets its input and output paths, loads the data, runs the attack and saves a JSON report and a PDF report. The PDFs are generated with ReportLab [49] straight from the attack output, with no manual editing. Each one records the time it was created and the software versions used.

### 5.2 Development and integration process

As a group, we followed an Agile approach based on SCRUM. It suited the project because each member built a separate component, but all the components still had to connect into one pipeline. We planned the work in two-week sprints with a shared backlog. Each sprint covered planning, design, implementation and testing. It ended

with a review with the supervisors and a short retrospective. Figure 5.1 shows this cycle.

Interface contracts were a main part of the process. The JSON Schema contracts for the exchanged files lived in the shared repository, and every exchanged file was checked against them. Version control kept all components in one repository, and continuous integration ran every component's tests on each push. Before the full experiment, a code review found the AES dataset key and the HMAC signing key written into source files. This was a problem because the repository was going to be published. We replaced both keys and moved them to environment files. A regression test now fails if an old key shows up in the source again.

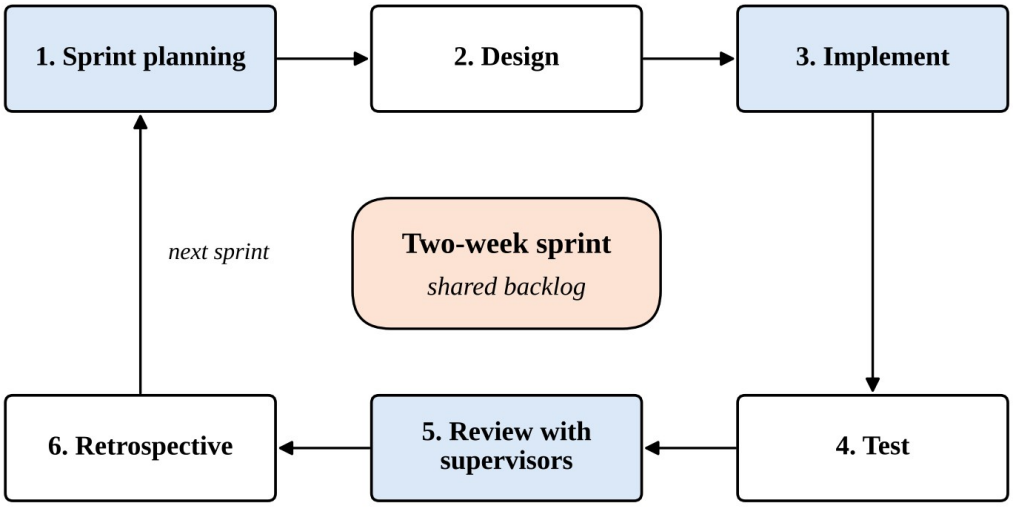



**Figure 5.1: Agile SCRUM cycle followed by the group**

Agile let the requirements change as results came in. For example, the encryption engine moved from CBC to GCM after the security audit. Another change was to the privacy classifier, after a label problem showed up in the first training runs.

### 5.2.1 Problems found during integration

Joining four components showed problems that no single member would have met alone. Table 5.3 lists the main ones, who found them and how they were fixed.

**Table 5.3: Problems found during development and integration**

| Problem | Found by | Fix |
|---|---|---|
| Encryption and signing keys written in source files | Code review | Both keys replaced and moved to environment files; regression test against old keys |
| Most positive labels were tiny pedestrians, and frames were stretched to a square | PI | Kept the aspect ratio and excluded frames with only small targets |
| Same frame numbers in different towns overwrote each other on merge | UE | Town prefix added to every file name before copying |
| Many frames have empty label files | UE | All scripts handle empty files as valid samples |
| Attack 4 could not load the saved models | RC | Model weights taken out of the full training state; exact model class shared by PI |
| k-anonymity check could not find the metadata columns | RC | Columns found by keywords such as weather, time, event and town |
| CBC cannot detect a changed ciphertext | DG audit | Encryption engine moved to GCM |
| Calibration budget reported as 1.8 instead of 12.6 | RC | Reporting code corrected; noise and statistics unchanged |

The label problem took the most time. In the first runs on real frames, both private and non-private models predicted the majority class for every input. We searched over learning rates, optimizers, batch sizes and ways to handle class imbalance. This search gave three useful findings. Adam was unstable under DP-SGD, so private training moved to SGD with momentum. Loss-based class weighting did not work well with per-sample clipping. Opacus replaces a weighted sampler, so balancing had to be done by duplicating frames. None of these changes fixed the main problem, though. Then we checked the labels. In 62.3% of positive frames, the largest pedestrian was smaller than about 8 × 8 pixels after resizing. Most positives were therefore label noise. On top

of that, the frames were being stretched into a 256 × 256 square. After keeping the aspect ratio and excluding frames with only small targets, the baseline test AUC rose from about 0.50 to 0.84. The early failure therefore came from data preparation and not from differential privacy.

The model loading problem came from an interface detail that no contract covered. Privacy Integration had saved full training states, which held the ε value and settings as well as the weights. The first version of Attack 4 assumed a fully connected network, while the real model is convolutional with group normalization. With strict=False, the private configurations could be loaded, and the results came within about 2% of those of Privacy Integration. Later, Privacy Integration shared the exact model class, so the architecture could be matched exactly.

Other problems were practical. The default output folders of Ultralytics are nested, so checkpoint paths had to be checked on disk before cross-evaluation. About 12 GB of KITTI images had to be downloaded and extracted on limited disk space. CPU-only training meant overnight runs, and the sleep settings of the computer had to be changed to stop training from being interrupted.

### 5.3 Deployment and operational configuration

The integrated solution runs as a series of command-line stages, joined by the pipeline runner. Keys are read from environment variables or a local .env file, so the same code can run on any machine. Privacy Integration also keeps all of its settings in one configuration file. In the intended deployment, the calibration stage runs on the data owner's machine. The capture and protection stage runs on the CARLA machine. The evaluation, training and audit stages run on machines that hold the encrypted dataset and the keys they need.

Monitoring relies on the evidence files. Every training run writes a per-step budget log and a signed report. The poisoning gate writes a PASS or FAIL verdict, and every attack writes a JSON and a PDF report. A master script in Risk and Compliance runs all six attacks in order and builds one combined compliance report. For recovery, the encrypted originals stay unchanged after decryption, and every run writes to its own folder. This means any stage can be repeated from its inputs.

Table 5.4 shows where each stage runs in the intended deployment, what it reads and what it writes. Only privatized, encrypted or signed files cross from one machine to another.

**Table 5.4: Stages of the deployed pipeline**

| Stage | Runs on | Reads | Writes |
|---|---|---|---|
| Calibration | Data owner's machine | Real aggregate statistics | calibration_stats.json; audit log for RC |
| Capture and protection | CARLA machine | Privatized statistics | Encrypted town folders, manifests and metadata |
| Decryption and gate | Analysis machine | Encrypted towns, manifests, labels | Decrypted images, clean labels, gate verdict |
| Detector training | Analysis machine | Decrypted images and labels; KITTI | Checkpoints, mAP results, FID |
| Private training and attacks | Analysis machine | Decrypted images and clean labels | Models, budget logs, signed reports, certificate |
| Independent audit | Auditor's machine | Signed evidence, models, town folders | JSON and PDF compliance reports |

To set up a stage, the operator installs the Python requirements of that component. The keys it needs go in a local .env file. Privacy Integration installs as a package, and its stages run from the command line. The Dataset Generation audit notebook runs on any machine without CARLA. The Risk and Compliance scripts only need their input and output paths set before they run.

During the project, we backed up the encrypted dataset and the keys to secure cloud storage.

### 5.4 Evidence of team execution

Our timeline started with topic selection and feasibility, then the literature review and gap analysis, proposal preparation, and requirement analysis and design. After that came component implementation, integration, testing and experiments. The last stage covered the research paper, the website, the final reports and the final presentation. Figure 5.2 shows the planned timeline, and Figure 5.3 shows how the work was broken down into four component streams.

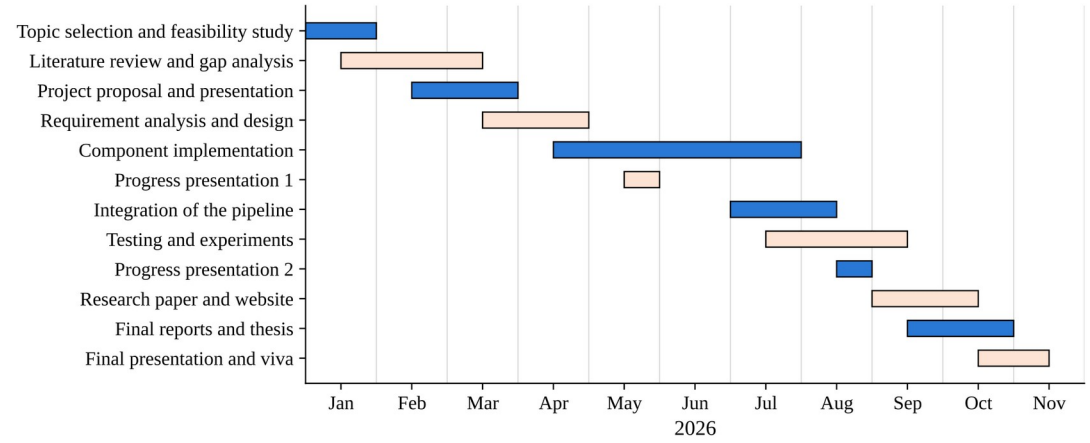



**Figure 5.2: Gantt chart of the project**

The main evidence of how the team carried out the work is listed below.

- **Version control and continuous integration.** We developed all components in one repository, and the tests ran on every push.
- **Supervisor reviews.** Each sprint ended with a review with the supervisors.
- **Progress reviews.** The proposal and progress review panels reviewed the project. We recorded their comments and responded to them (Section 6.4).
- **Cross-component checks.** Risk and Compliance used its own code to verify the outputs of Privacy Integration and Dataset Generation. Its reports record this joint work.

Each of the four streams in the work breakdown covered the same five kinds of work: research, implementation, testing, result collection and documentation. This made it easy to compare progress across members at each review. The integration outputs are evidence in their own right. The signed reports, budget logs, manifests, gate verdicts and attack reports were each produced by one member and checked by another. The problems in Table 5.3 were found and fixed through this shared work.

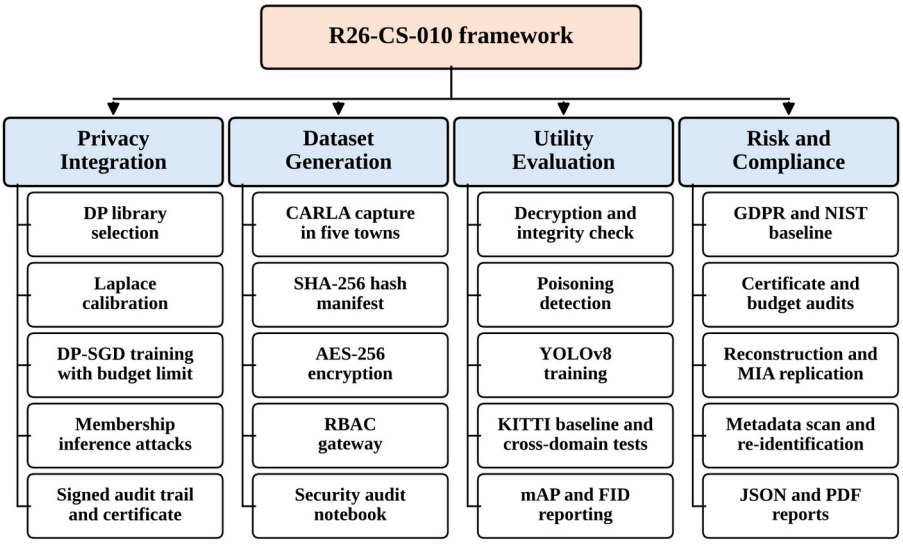



**Figure 5.3: Work breakdown structure of the project**

# 6 INTEGRATED TESTING EVALUATION AND VALIDATION

## 6.1 Evaluation framework and success criteria

The group evaluation asked four questions, one for each subproblem:

- **EQ1 (SP1):** Does differential privacy at calibration and training stop membership inference, and at what cost?
- **EQ2 (SP2):** Are the generated files confidential and tamper-evident from the moment of capture?
- **EQ3 (SP3):** Is the protected synthetic data still useful for training a detector, and are its labels untouched?
- **EQ4 (SP4):** Do the privacy controls hold when an independent party attacks them?

We used the group objectives in Table 2.3 as the acceptance criteria. We also used three baselines. A non-private model was the positive control for every membership inference attack. So if a private model scored near chance, a broken attack could not be the reason. A detector trained on real KITTI images [1] was the utility baseline. A label file with planted defects was the control for the poisoning gate.

The main dataset was the five-town CARLA dataset of 10,000 annotated frames. Each frame has an RGB image, a depth map, a segmentation image and a class mask. The real baseline was the KITTI training split of 7,481 images. A separate Town01 capture of 6,093 images tested the GCM engine on real output. No human participants took part. We report privacy results as means and standard deviations over three seeds, with paired t-tests against same-seed baselines. Utility is reported as mAP50, mAP50-95 and utility retention. Utility retention is the synthetic-to-real mAP50 divided by the real-to-real mAP50. The individual theses describe each method in full.

Each component used the datasets in a slightly different way. Table 6.1 shows how the data was split and used for each part of the evaluation.

**Table 6.1: Datasets and splits used in the evaluation**

| Dataset | Use | Size or split |
|---|---|---|
| Five-town CARLA dataset | Main synthetic dataset for all components | 10,000 frames, 800 × 600 pixels, four images per frame |

Table 6.2 lists the measures we used. Most of them come from the cited literature. We chose them so that each evaluation question has at least one measure that a reader outside the team can repeat.

**Table 6.2: Measures used in the evaluation**

| Measure | Question | Meaning |
|---|---|---|
| Test AUC | EQ1 | How well the privacy classifier separates frames with and without pedestrians or cyclists |
| Attack AUC and attack success | EQ1, EQ4 | How well an attacker separates training members from non-members; 0.5 and 50% mean random guessing |
| TPR at 1% and 5% FPR | EQ1 | Whether the attacker can identify a few members with high confidence |
| Final logged ε | EQ1, EQ4 | The privacy budget actually spent by a training run |
| Chi-square and bit difference | EQ2 | Whether the ciphertext looks like random data |
| Tamper and folder checks | EQ2 | Whether changed files are rejected and stored files are unreadable |
| mAP50 and mAP50-95 | EQ3 | Detection quality at 50% IoU and averaged from 50% to 95% IoU |
| Utility retention | EQ3 | Synthetic-to-real mAP50 divided by real-to-real mAP50 |
| FID | EQ3 | Distance between the synthetic and real image sets |
| Re-identification rate | EQ4 | Share of synthetic samples that match a reference sample above a threshold |

## 6.2 Integration and end to end testing

We tested at four levels.

- **Interface tests.** Every exchanged file was checked against its JSON Schema contract. Before any training, Risk and Compliance re-hashed the decrypted dataset, and the poisoning gate had to produce the clean labels.
- **System tests.** An end-to-end pipeline test ran through continuous integration on every push, along with the unit tests of every component. Privacy Integration alone has 107 unit tests, and all of them passed.

- **Security tests.** The Dataset Generation security audit tested round-trip correctness, nonce uniqueness, ciphertext randomness, bit difference and tamper detection. The six attacks from Risk and Compliance tested the privacy controls of the whole pipeline.
- **Performance and usability.** In a single run, encrypting a 4 KB test file took about 15 ms, and decrypting it took about 26 ms. So protection did not noticeably slow the pipeline down. The Risk and Compliance PDF reports were written so that a reader without technical training can follow them.

Table 6.3 lists the main integration and system tests, with the result of each. Most of them check a hand-off between two components. Some check a property that a later component depends on, such as the budget limit or the repeatability of a run.

**Table 6.3: Integration and system test results**

| Test | Expected result | Actual result |
|---|---|---|
| Decryption with manifest check | Every decrypted file matches its digest | 40,000 of 40,000 files matched |
| Poisoning gate on the full dataset | PASS with no problems | PASS; 15,084 objects checked |
| Poisoning gate on planted defects | Defects are flagged | 3 problems flagged |
| Calibration reaches the simulator | Town01 profile equals the privatized file | Matched field by field |
| Budget limit | Training halts at the limit | Halted in a low-limit test |
| Per-step budget logging | One row per step | 3,090 rows per run |
| Repeatability | Same seed gives the same result | Same loss and test AUC |
| Signed report tampering | Changed report is rejected | Rejected by PI and by RC |
| Privacy Integration unit tests | All pass | 107 of 107 passed |

We did not run a formal user acceptance test with external users. Section 6.4 describes how the solution was reviewed instead.

### 6.3 Overall results

Table 6.4 summarizes the main result of each component. The sections after it give the group-level evidence behind each row. The detailed experiments are in the individual theses.

**Table 6.4: Summary of the main results of each component**

| Component | Main result | Answer to its question |
|---|---|---|
| Privacy Integration | Attack AUC fell from 0.639 (no DP) to 0.504 to 0.515 (ε = 1, 3, 8); test AUC fell by 0.16 to 0.21 | EQ1: yes, at a measured cost |
| Dataset Generation | All five audit tests passed; a single changed byte was rejected; 0 of 6,093 captured files readable | EQ2: yes |
| Utility Evaluation | mAP50 of 0.3655 on synthetic images; 10.46% retention on KITTI; FID 115.99; poisoning gate PASS | EQ3: useful within the synthetic domain, weak on real images |
| Risk and Compliance | All six attacks passed; replication within about 2%; 0% re-identification; one reporting error found | EQ4: yes |

#### 6.3.1 Privacy at calibration and training

The calibration profile in the Town01 configuration matched the privatized file field by field, so the simulator used the privatized values. The delivered Town01 metadata shows the same thing at the frame level. Of its 2,000 frames, 1,248 are clear, 270 are cloudy and 482 are rainy, with no fog. These shares are exactly the privatized weather values of 0.624, 0.135, 0.241 and 0.000. The time-of-day labels follow the privatized values in the same way, to within rounding.

For training, Table 6.5 gives the mean over three seeds for each configuration. Figure 6.1 plots the two main measures.

**Table 6.5: Privacy and utility by configuration (mean ± standard deviation over three seeds)**

| Configuration | Target ε | Test AUC | Attack AUC | Attack success | Loss gap |
|---|---|---|---|---|---|
| No DP | none | 0.829 ± 0.009 | 0.639 ± 0.013 | 56.7% | 0.360 |
| Config A | 1.0 | 0.621 ± 0.045 | 0.504 ± 0.002 | 50.3% | 0.001 |
| Config B | 3.0 | 0.655 ± 0.035 | 0.515 ± 0.002 | 51.6% | 0.013 |

| Configuration | Target ε | Test AUC | Attack AUC | Attack success | Loss gap |
|---|---|---|---|---|---|
| Config C | 8.0 | 0.668 ± 0.020 | 0.506 ± 0.007 | 50.8% | 0.026 |

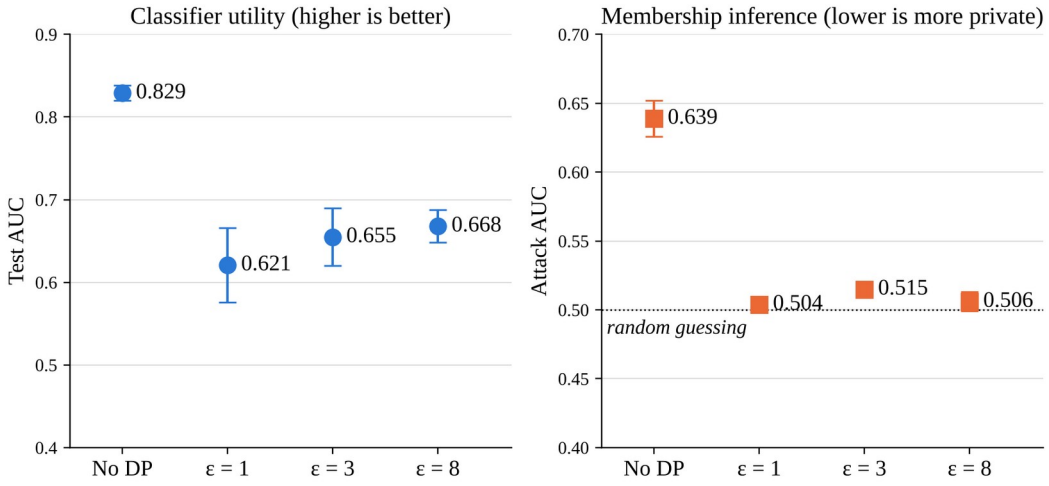



**Figure 6.1: Test AUC and attack AUC by privacy configuration (mean and one standard deviation over three seeds)**

The attack worked against the non-private baseline. Its mean AUC was 0.639 and its success rate was 56.7%. On training images, the baseline loss was 0.36 lower than on unseen images, and this overfitting is the signal the attack uses. Against every private configuration, the attack AUC fell to between 0.504 and 0.515. The success rate fell to between 50.3% and 51.6%, which is close to random guessing. The single runs did not overlap either. The nine private runs had an attack AUC between 0.499 and 0.516, and the three baseline runs were between 0.625 and 0.650. The cost was a fall in test AUC from 0.829 to between 0.621 and 0.668.

We ran paired t-tests against the baseline trained with the same seed. For every private configuration, both drops were significant at the 0.05 level, as Table 6.6 shows. With only three seeds, the differences between the three private configurations were not significant.

**Table 6.6: Paired t-tests of each private configuration against the baseline (three seeds)**

| Configuration | Drop in test AUC | p-value | Drop in attack AUC | p-value |
|---|---|---|---|---|
| Config A | 0.208 | 0.0099 | 0.134 | 0.0025 |
| Config B | 0.174 | 0.0084 | 0.124 | 0.0032 |

| Configuration | Drop in test AUC | p-value | Drop in attack AUC | p-value |
|---|---|---|---|---|
| Config C | 0.160 | 0.0031 | 0.132 | 0.0031 |

A second attack checked this result. The loss-threshold attack [21] needs no shadow model. It treats a lower loss on the target model as a sign of membership. It used the same 1,179 members and 1,179 non-members, and every run reproduced the test AUC recorded at training, so the splits matched. Table 6.7 shows the results, and Figure 6.2 compares the two attacks.

**Table 6.7: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR)**

| Configuration | Attack AUC | TPR at 1% FPR | TPR at 5% FPR |
|---|---|---|---|
| No DP | 0.610 ± 0.026 | 0.007 ± 0.007 | 0.057 ± 0.012 |
| Config A | 0.501 ± 0.007 | 0.015 ± 0.002 | 0.053 ± 0.011 |
| Config B | 0.503 ± 0.012 | 0.016 ± 0.003 | 0.055 ± 0.013 |
| Config C | 0.506 ± 0.012 | 0.014 ± 0.006 | 0.053 ± 0.009 |

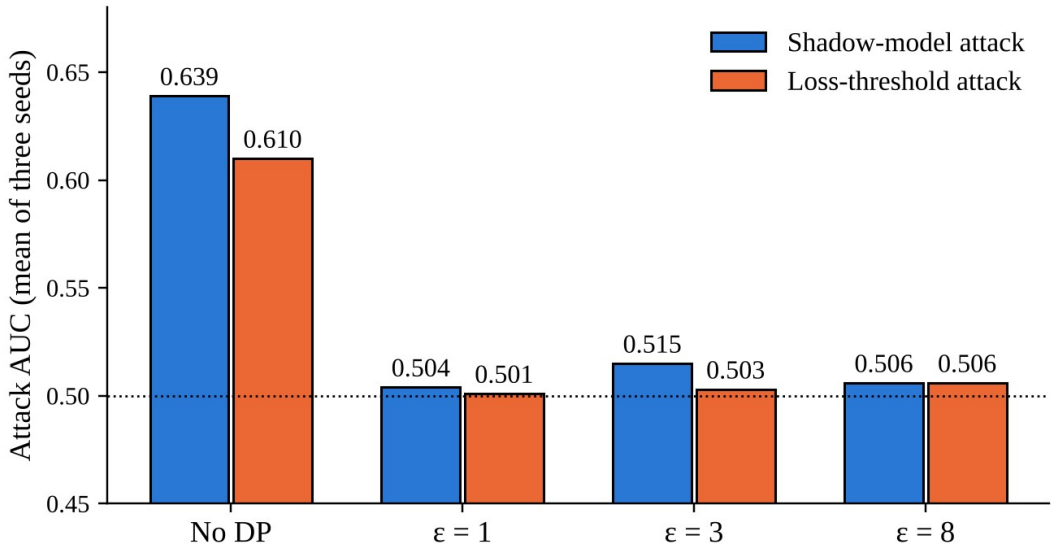



**Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks**

The loss-threshold attack gave an AUC of 0.610 against the baseline. For every private run, it fell to between 0.491 and 0.518. The paired drop of 0.10 to 0.11 was significant for each private configuration ($p = 0.025$, 0.033 and 0.035). At low false-positive rates, however, the attack was at chance level for every configuration, including the baseline.

These values are rough, because a 1% FPR allows only about 12 false alarms. Still, they show what the baseline leaked. It was a small average shift in loss, spread across many images. Even without DP, no single frame could be picked out with high confidence.

The three budgets gave almost the same result against the attacks. This is true even though $\varepsilon = 8$ is a much weaker formal bound than $\varepsilon = 1$. This agrees with earlier studies [22], [23]. Realistic attacks fall well short of the formal bound. Most of the utility cost therefore comes from using DP-SGD at all, and little comes from the exact budget.

### 6.3.2 Protection of generated files

All security audit tests ran in the audit notebook, using the same SecurityEngine code as the pipeline and a fixed 32-byte test key. Table 6.8 summarizes the results, and Figure 6.3 shows four of them together.

**Table 6.8: Summary of the Dataset Generation security audit**

| Property | Evidence | Outcome |
|---|---|---|
| Round-trip correctness | Bytes identical after decryption | Pass |
| Nonce uniqueness | 30 of 30 unique nonces and ciphertexts | Pass |
| Byte randomness | Chi-square 272.59 (critical value 293.25) | Pass |
| Bit difference | 50.0% of bits differ | Pass |
| Tamper detection | MAC check failed on the changed file | Pass |
| Real folder | 0 of 6,093 encrypted files readable | Pass |

Encrypting a 4 KB file and decrypting it again gave back the exact original bytes. Encryption took 14.914 ms and decryption took 25.931 ms, including file reading and writing. These times come from one run only, so they are a rough guide. When the same file was encrypted 30 times, the result was 30 different nonces and 30 different ciphertexts. The nonce is a random 128-bit value. For the 6,093 files of the real capture, the chance that any two nonces collide is about $6{,}093^2 / 2^{129}$. That is close to $5 \times 10^{-32}$.

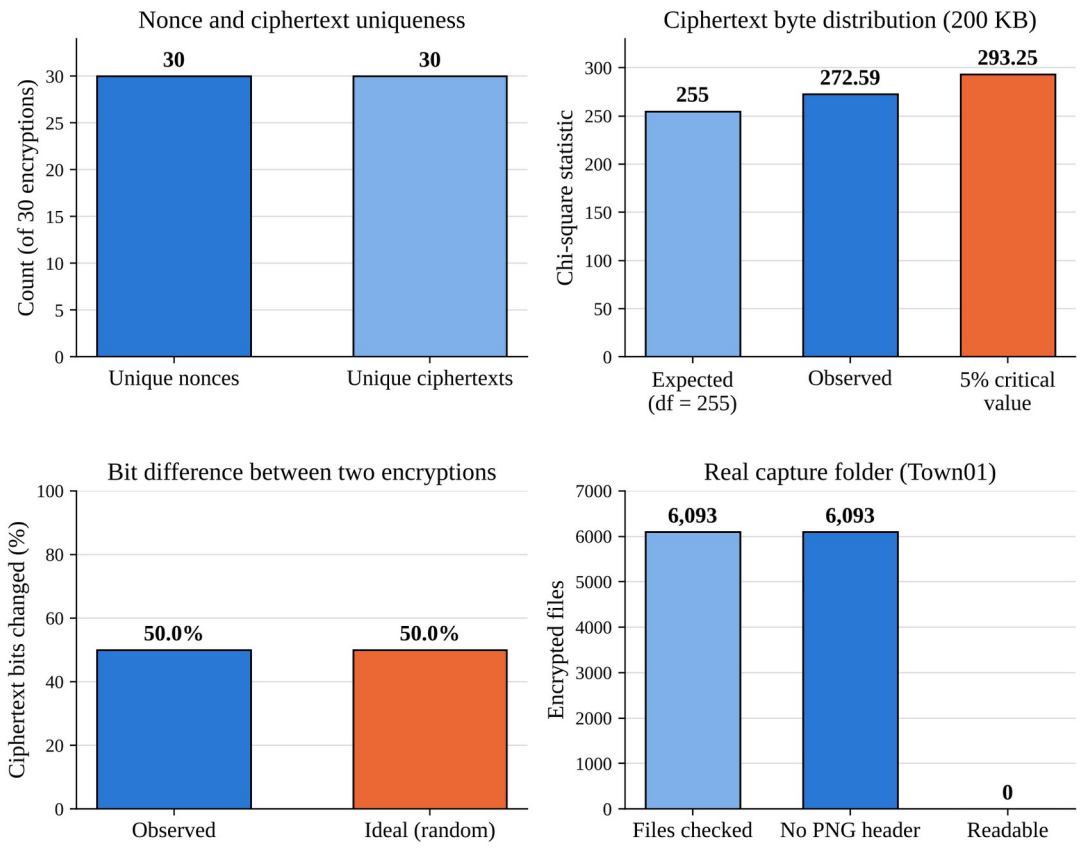



**Figure 6.3: Results of the Dataset Generation security audit**

For the randomness test, a 200,000-byte file was encrypted. The count of each of the 256 byte values was then compared with the expected count of 781.25. The chi-square statistic was 272.59 with 255 degrees of freedom. This is below the 5% critical value of about 293.25, with a p-value of about 0.21. The test therefore cannot tell the ciphertext apart from random bytes. In the bit difference test, two 4 KB inputs that differ in one bit gave ciphertexts in which 16,384 of 32,768 bits differed. We read this result with care. In GCM, a one-bit change under the same nonce changes only one bit of the ciphertext. The 50.0% comes from the fresh nonce used for each encryption, which is the property the pipeline needs.

For the tamper test, one byte of an encrypted file was flipped with an XOR of 0xFF. Decryption failed with the error "MAC check failed", and no output file was written. Next, the audit scanned every .png.enc file in the real Town01 output folder. None of the 6,093 files started with a PNG signature, so none could be read without the key. Each GCM file costs only 32 extra bytes, or about 190 KB for the whole capture.

### 6.3.3 Utility of the synthetic data

Utility Evaluation decrypted all 40,000 image files of the five towns. Every SHA-256 digest matched its manifest entry. The poisoning gate then checked 15,084 object instances across the five towns. It found zero problems and no significant difference in class proportions between towns, so the verdict was PASS. On the copy with two planted defects (x_center = 1.5 and class 7), the gate flagged three problems. The gate does find real tampering, then, and the dataset was not changed before training.

Utility Evaluation studied the label files before any training. The classes turned out to be very imbalanced. There are 11,008 cars, 2,424 pedestrians and 1,652 cyclists, a ratio of about 5 to 1 between cars and pedestrians. Also, 32.6% of frames contain no objects. Pedestrians and cyclists are often very small: 86.4% of pedestrian instances and 67.3% of cyclist instances are smaller than 32 × 32 pixels. Figure 6.4 shows these findings. They explain why we report per-class results throughout.

Table 6.9 gives the results of the four cross-domain experiments, and Figure 6.5 compares them.

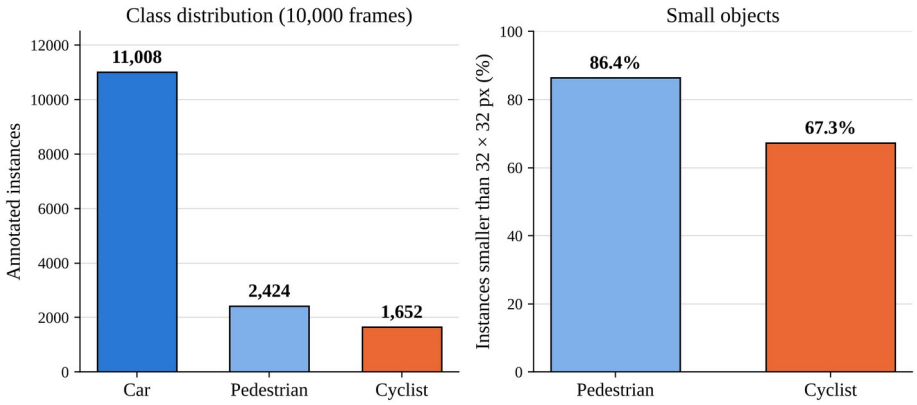



**Figure 6.4: Class distribution and share of small objects in the synthetic dataset**

**Table 6.9: Cross-domain evaluation results**

| Experiment | mAP50 | mAP50-95 | Car AP50 | Pedestrian AP50 | Cyclist AP50 |
|---|---|---|---|---|---|
| Synthetic to Synthetic | 0.3655 | 0.2273 | 0.6076 | 0.1793 | 0.3096 |
| Synthetic to Real (KITTI) | 0.0889 | 0.0343 | 0.2614 | 0.0051 | 0.0001 |
| Real (KITTI) to Real (KITTI) | 0.8500 | 0.5531 | 0.9519 | 0.7769 | 0.8213 |

| Experiment | mAP50 | mAP50-95 | Car AP50 | Pedestrian AP50 | Cyclist AP50 |
|---|---|---|---|---|---|
| Real (KITTI) to Synthetic | 0.0507 | 0.0199 | 0.1273 | 0.0079 | 0.0171 |

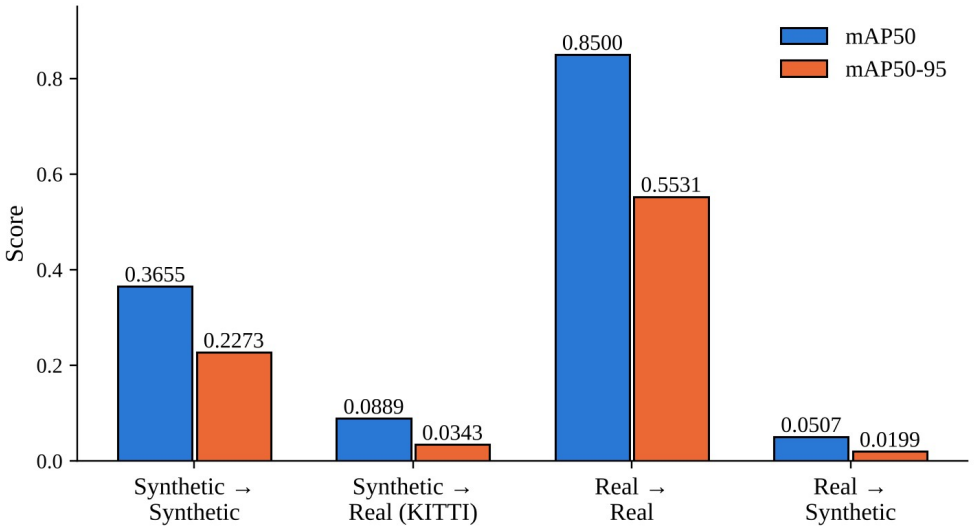



**Figure 6.5: mAP50 and mAP50-95 of the four cross-domain experiments**

On its own test split, the synthetic model reached an mAP50 of 0.3655 and an mAP50-95 of 0.2273. The synthetic-to-real mAP50 of 0.0889 is 10.46% of the real-to-real mAP50 of 0.8500. This utility retention is the measured cost at the data level. The loss is uneven across classes, as Figure 6.6 shows. Cars kept about 27.5% of their real-data AP50 (0.2614 against 0.9519), while pedestrians (0.0051) and cyclists (0.0001) almost disappeared. The reverse transfer, from real to synthetic, gave an mAP50 of only 0.0507, so the domain gap works in both directions. The four results fall in a sensible order. Real to real is best, synthetic to synthetic comes next and the two cross-domain pairs are lowest. This order supports the internal validity of the protocol.

The FID between the synthetic and KITTI test images was 115.99, with 1,000 images from each set. Table 6.10 gives the scale we used to read it.

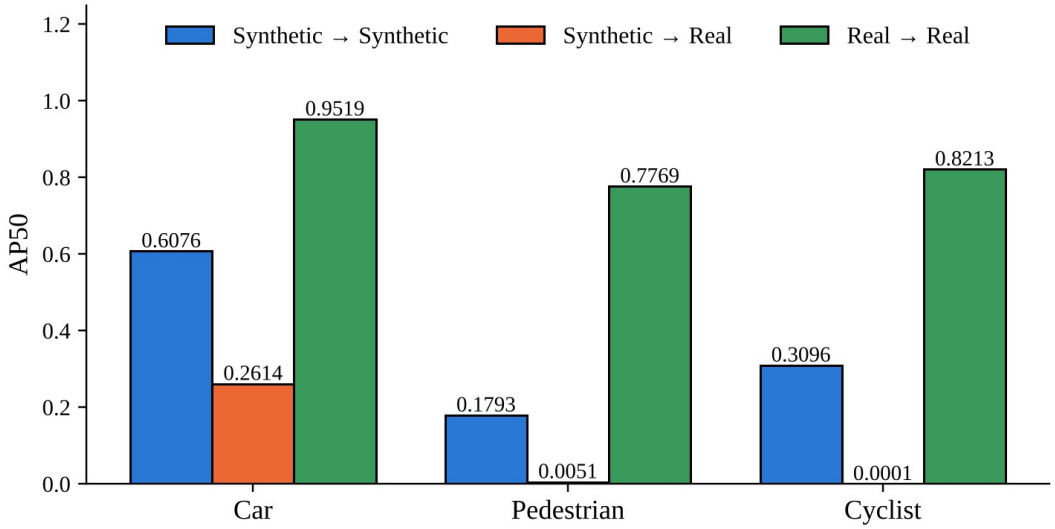



**Figure 6.6: Per-class AP50 for training and testing within and across domains**

**Table 6.10: FID interpretation scale**

| FID range | Interpretation |
|---|---|
| Below 50 | Reasonable similarity |
| 50 to 200 | Moderate difference |
| Above 200 | Very different |

A score of 115.99 falls in the moderate range. The utility result, however, was much worse than moderate. FID alone does not predict how well objects will be detected, so mAP and FID should be read together.

### 6.3.4 Independent verification

Risk and Compliance ran all six attacks. Table 6.11 lists what each attack tested and whether the result matched the expected outcome.

**Table 6.11: Results of the six independent attacks**

| Attack | What was tested | Result | Outcome |
|---|---|---|---|
| Attack 1 | All signed reports pass verification | 4 of 4 reports verified; signatures matched | Passed |
| Attack 1 | Tampering with a report is detected | Changing ε from 0.990572 to 0.01 was caught | Passed |
| Attack 2 | Reported ε matches the training log | All 3 configurations within 0.0001 | Passed |
| Attack 2 | ε only increases during training | Never decreased at any logged step | Passed |

| Attack | What was tested | Result | Outcome |
|---|---|---|---|
| Attack 2 | ε never exceeds the ceiling of 10 | Peaks of 0.99, 3.00 and 7.99 | Passed |
| Attack 3 | Noise ratio near 1.0 | Ratio of 0.94, inside 0.5 to 1.5 | Passed |
| Attack 4 | MIA fails against DP models | AUC between 0.50 and 0.53 for all DP models | Passed |
| Attack 4 | Results match those of Privacy Integration | Within about 2% for all DP configurations | Passed |
| Attack 5 | No sensitive data in metadata | Zero findings in 10,015 files | Passed |
| Attack 6 | Re-identification rate below 5% | 0% for camera, LiDAR and trajectory | Passed |

For the budget audit, Config A ended at 0.990572 (target 1.0), Config B at 2.99984 (target 3.0) and Config C at 7.995129 (target 8.0). Each is slightly below its target, because Opacus picks σ so that the budget at the final step does not pass the target. Figure 6.7 compares the final values with the targets and the limit.

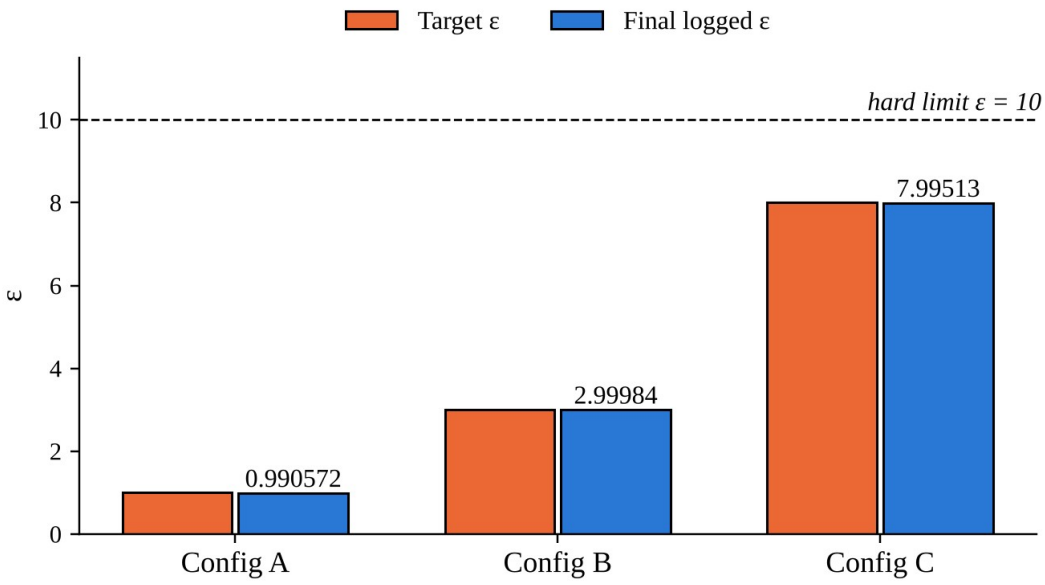



**Figure 6.7: Target and final logged ε of the three private configurations**

The reconstruction attack agrees with a correct Laplace mechanism. Across the five scalar fields, the mean absolute difference between the published and source values was 0.6246, against an expected 0.6667. The ratio of 0.94 is close to the ideal of 1.0. The largest effect was on intersections per kilometer, where the published value was

0.28 against a source value of 1.21, a distortion of 76.8%. An attacker who sees only the published value cannot work out the source value.

The independent membership inference attack gave AUC values of 0.5205 for Config A, 0.5284 for Config B and 0.5000 for Config C. Privacy Integration had reported 0.5044, 0.5144 and 0.5131 for the same configurations. Against the non-private baseline, the independent attack gave an AUC of 0.65, close to the mean of 0.639 reported by Privacy Integration. We needed this positive control. If every model, including the unprotected one, had scored 0.50, we could not tell whether privacy worked or the attack was broken.

The metadata scan covered 30,000 encrypted images and 10,015 metadata files across the five towns, and it found nothing sensitive. The frame analysis showed that no town starts at frame zero. Town01 starts at frame 224, Town02 at 32, Town03 at 112, Town04 at 64 and Town05 at 3136. These gaps appear because CARLA drops some frames while it warms up, so they reveal nothing private. Table 6.12 shows the re-identification results, and Figure 6.8 compares the highest similarity found with each threshold.

**Table 6.12: Re-identification results by modality**

| Modality | Threshold | Samples | Re-ID rate | Highest similarity | Verdict |
|---|---|---|---|---|---|
| Camera | 0.85 | 200 | 0.0% | 0.1635 | Pass |
| LiDAR | 0.80 | 200 | 0.0% | 0.3464 | Pass |
| Trajectory | 0.75 | 200 | 0.0% | 0.4745 | Pass |
| Overall | none | 600 | 0.0% | 0.4745 | Pass |

Finally, Table 6.13 links each GDPR article to the attack result that supports it. The same controls were mapped to the five functions of the NIST Privacy Framework: Identify, Govern, Control, Communicate and Protect [46].

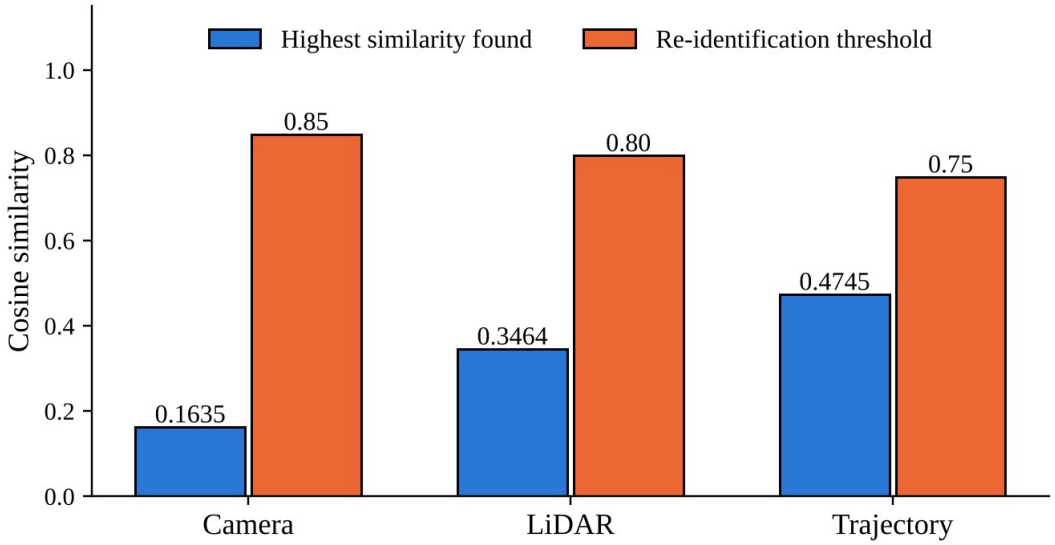



**Figure 6.8: Highest cosine similarity found against the re-identification threshold**

**Table 6.13: GDPR compliance mapping with attack evidence**

| GDPR article | Legal requirement | Result | Evidence |
|---|---|---|---|
| Art. 5(1)(b) Purpose limitation | Data used only for the purpose it was collected for | Pass | Calibration log shows only 7 aggregate queries; no individual records accessed |
| Art. 5(1)(c) Data minimization | Process the least data needed | Pass | Attack 3: source values not recoverable; Attack 6: zero re-identification |
| Art. 5(2) Accountability | Controller must be able to show compliance | Pass | Attack 1: tamper-evident audit trail verified independently |
| Art. 17 Right to erasure | No data kept beyond its purpose | Pass | Attack 5: no individual-level data in metadata |
| Art. 25 Privacy by design | Privacy built into the system | Pass | Attack 2: DP enforced at every step; Attack 4: MIA fails on all DP models |
| Art. 32 Security of processing | Suitable technical measures | Pass | AES-256 encryption with SHA-256 manifests; Attack 5: no metadata leakage |

### 6.3.5 Combined view

Table 6.14 puts the two cost measurements side by side.

**Table 6.14: Cost of privacy at the data level and the model level**

| Level | Component | Finding |
|---|---|---|
| Data | Utility Evaluation | A detector trained on the synthetic data keeps 10.46% of the real-data mAP50 when tested on real KITTI images |
| Model | Privacy Integration | DP-SGD lowers test AUC by 0.16 to 0.21 and stops membership inference (attack AUC from 0.639 to about 0.50) |

At the model level, the cost is moderate and well understood. At the data level, the gap between synthetic and real images is large. We cannot blame this gap on differential privacy alone. The dataset was produced at one calibration budget, and we had no non-private dataset to compare against. Differences in rendering, object size and image shape add to it.

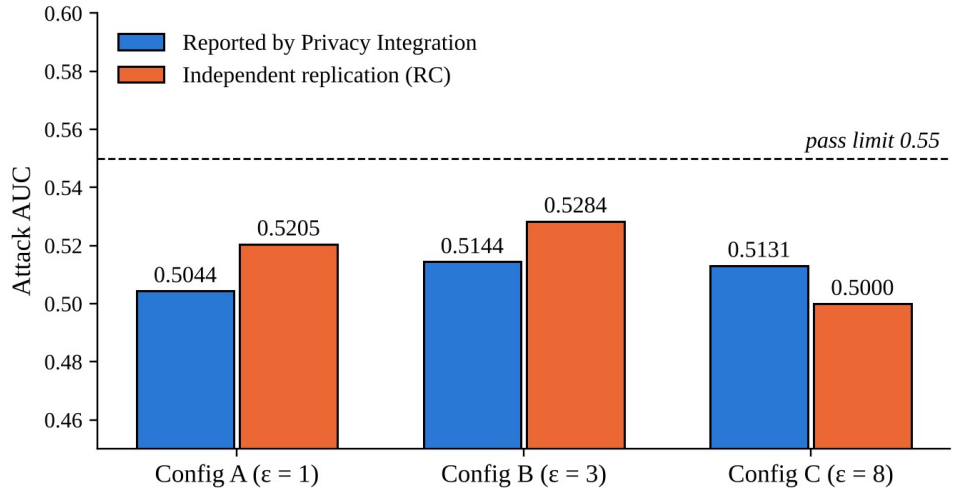



**Figure 6.9: Attack AUC reported by Privacy Integration and found by the independent replication**

Integration also caught an error that no single component would have found on its own. The composition check in Risk and Compliance found that the calibration budget had been reported as 1.8. That is the value for each of the seven queries. The sequential total, however, is $7 \times 1.8 = 12.6$. The noise itself was always correct, so the fix only changed the report, and no capture or training had to be repeated. We found this error because one component signed its evidence and another checked it with separate code.

Figure 6.9 shows a second example of this cross-check, where the independent attack reproduced the Privacy Integration results.

### 6.4 Stakeholder and user validation

Three groups reviewed the complete solution. The supervisors reviewed the work at the end of each sprint. They also helped us narrow the scope to four components we could build and test. Risk and Compliance was the internal user of the evidence from the other components, and it verified that evidence without their help. The progress review panels evaluated the solution at the proposal and progress stages.

At the second progress review (PP2), the panel gave one comment per component. Table 6.15 lists the comments, our response and the concern that remains.

**Table 6.15: Comments of the second progress review panel and the team's responses**

| Component | Panel comment | Response | Remaining concern |
|---|---|---|---|
| PI | Even with differential privacy, the reduction of attacks is not significant | The drop is statistically significant ($p \leq 0.0032$), but small in absolute terms, because the baseline leaked only moderately (success 56.7%) | A stress test against a more strongly overfitted baseline |
| DG | How is the original file removed? Is it using crypto-shredding? | The GCM engine deletes the plaintext with a normal file delete after encryption. This is not crypto-shredding and may leave recoverable data on disk | Encrypt in memory so that plaintext is never written; per-batch keys for crypto-shredding |
| UE | How was data poisoning detection validated? | Two defects were planted in a separate label file (x_center = 1.5 and class 7) and were flagged, while the clean dataset passed | The statistical check has not yet been tested with planted label changes |
| RC | Metadata leakage findings are 0 | Zero findings are expected for simulator data, but a scan with no positive control cannot show that it would find a leak | Plant known test values in a copy of the metadata and show that the scan finds them |

These comments led to one change across the whole solution. Each component now reports two things: the result of its test, and whether the test was shown to detect a real problem. The open items are listed as future work in Section 8.3.

### 6.5 Achievement of common objectives

Table 6.16 maps the evidence to each group objective.

**Table 6.16: Achievement of the group objectives**

| No. | Objective | Evidence | Status |
|---|---|---|---|
| GO1 | Training budget within ε = 10 | Final ε of 0.991, 3.000 and 7.995; logs match reports within 0.0001 | Fully achieved |
| GO2 | Attack success at or below 55% | 50.3% to 51.6% with DP, 56.7% without; independent replication within about 2% | Fully achieved |
| GO3 | Perception within 15% of real data | 10.46% of the real-data mAP50 on real images | Not achieved |
| GO4 | Calibration privatized within ε = 2 | Laplace noise on all seven queries; simulator used the privatized values; sequential total of 12.6 | Partly achieved |
| GO5 | Every generated file protected | SHA-256, AES-256 and RBAC; all audit checks passed; 100% of manifests matched after decryption | Fully achieved |
| GO6 | Labels checked for poisoning | Clean data passed; planted defects flagged | Fully achieved |
| GO7 | Independent verification and legal mapping | Six attacks passed; six GDPR articles and five NIST functions mapped | Fully achieved |

GO3 was not achieved. The detector trained on synthetic data worked inside the synthetic domain. On real images, though, it lost most of its performance, especially for pedestrians and cyclists. GO4 was only partly achieved. Each query was below the limit, but their sequential total was not.

Each of the five fully achieved objectives has evidence from more than one source. For GO1, the budget logs of Privacy Integration were checked by the independent budget audit. GO2 is supported by two attacks from Privacy Integration and a third from Risk and Compliance. The security audit of Dataset Generation and the manifest checks of Utility Evaluation both back GO5. For GO6, the gate passed the clean data and flagged the planted defects. GO7 has six attack reports and the mapping in Table 6.13 behind it. The causes of the two objectives that were not fully met are explained above, and Section 8.3 lists the work needed to meet them.

### 6.6 Integrated limitations and threats

**Dependencies between components.** The delivered dataset was encrypted in CBC mode (AES-256-CBC). CBC has no built-in tamper detection, so the integrity of the dataset depends on the SHA-256 manifests. The GCM engine was tested on a separate Town01 capture. The dataset was generated at one calibration budget, so we could not measure how much of the utility gap came from calibration noise. Testing a new budget would need a new capture.

**Validity threats.** The calibration source was reference statistics from CARLA runs, not a real dataset. Three seeds were enough to separate private from non-private models, but not to separate the three budgets from each other. Our membership inference attacks were realistic, but they were not the strongest known [6]. CARLA and KITTI images have different shapes, which affects the measured utility gap. CARLA images are 800 × 600 pixels, while KITTI images are about 1242 × 375. Training detectors on a CPU also limited how many extra experiments we could run. Several tests lack a positive control, as the review panel pointed out (Table 6.15).

**Scope of the security tests.** A passing statistical test shows that no weakness was found in that property. It does not prove that a system cannot be broken, and it does not replace formal cryptanalysis. The audit inputs were pseudo-random bytes, and the audit notebook covers the encryption layer only. Tests of the manifest check and of RBAC denial cases are not yet part of the notebook. The timing results come from a single run and are not a full benchmark.

**Unresolved integration risks.** The security audit uses a fixed test key. The keys are kept in local files, without a key management service. The RBAC gateway has no user authentication and keeps no persistent access log. Opacus secure mode was turned off, because the library it needs does not support current PyTorch versions. At first, the independent attack loaded the private models with a relaxed setting, before the exact model class was shared.

**Generalizability.** The results come from one simulator, five towns, one privacy classifier and one detector family. They may not hold for other simulators, larger models or other kinds of sensor data.

# 7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

## 7.1 Value proposition and beneficiaries

The framework could be offered as a privacy and security layer for teams that build perception models with simulated data. Such a team would receive synthetic data along with a record that another party can check. This record shows where privacy was applied and how the files were protected. It also shows what privacy cost in utility and which independent tests passed. With this record, a team can use real driving statistics in its simulation and still show that the statistics, files and models were protected.

The beneficiaries are the stakeholders in Table 1.1. Developers get data they can use without a long privacy review for every dataset. Road users are less likely to have their faces, plates or routes exposed. Data protection officers and regulators get evidence they can read and verify. Researchers get datasets they can share with a clear record.

Table 7.1 sets out the value for each group of customers and users in more detail. The value differs because each group uses a different part of the output.

**Table 7.1: Value of the framework for each customer or user group**

| Group | Value they get |
|---|---|
| Autonomous vehicle developers | Synthetic data built from privatized statistics, with a signed privacy record they can hand to their data protection officer |
| Data protection officers and regulators | Plain-language PDF reports that explain each test, its result and the GDPR articles it supports |
| University research labs | Encrypted datasets with manifests that can be shared with partners and checked on arrival |
| Simulation service providers | A way to accept calibration data from clients without receiving raw recordings |
| Road users and the public | Less exposure of faces, plates and routes in training pipelines and trained models |

Many privacy claims in this area are hard to check from the outside. A framework that signs its evidence and invites independent attacks makes it easier for others to question a claim. That helps build trust in synthetic data as a whole.

### 7.2 Market alternatives and differentiation

We target autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers. Many of them want real statistics to make their simulations realistic. However, privacy rules and contracts stop them from sharing raw recordings.

Each existing alternative covers only part of this need. Face and plate blurring removes visible identifiers, but it does nothing to protect the trained model. Simulators such as CARLA and NVIDIA DRIVE Sim produce realistic data. However, our project proposal did not find any formal privacy guarantee for their calibration step. Private generative models such as DP-GAN were only tested on small benchmarks [17]. Training libraries such as Opacus [15] apply DP-SGD but do not test the result. Auditing tools such as ML Privacy Meter [38] measure leakage, but they neither apply privacy nor protect files.

What makes our framework different is that it joins these steps into one pipeline. It privatizes the calibration channel, protects every file from capture, measures utility against real data and has every claim checked by separately written attacks. None of the alternatives above does all four.

Table 7.2 compares these alternatives with the framework. It lists what each alternative covers and what it leaves out.

**Table 7.2: Existing alternatives compared with the framework**

| Alternative | What it covers | What it leaves out |
|---|---|---|
| Face and plate blurring | Removes visible identifiers from real images | Trained models, calibration data and file protection |
| Simulators such as CARLA and NVIDIA DRIVE Sim | Realistic synthetic scenes with automatic labels | A formal privacy guarantee for the calibration step |
| Private generative models such as DP-GAN | Formal privacy during generation | Tests on driving data; file protection; independent checks |
| DP training libraries such as Opacus | DP-SGD and budget accounting | Testing whether the budget actually stops attacks |
| Auditing tools such as ML Privacy Meter | Measuring the leakage of a trained model | Applying privacy and protecting files |

| Alternative | What it covers | What it leaves out |
|---|---|---|
| Our framework | Calibration privacy, file protection, utility measurement and independent checks in one pipeline | Closing the synthetic-to-real gap; production key management |

Each part on its own uses known methods. Our advantage is in how the parts are joined. The file contracts, the signed evidence and the attack suite were built to work together, and every control comes with a measured cost. Another team would need time to rebuild all of this.

### 7.3 Business model and feasibility

The most realistic business model is an open-source core with paid services. The core could be released as an open-source toolkit. This lowers the barrier for research labs and builds trust in the method. Companies could then pay for integration with their own simulators and pipelines, and for support and training. A hosted version could run the privacy audits and produce compliance reports as a service. Universities could use academic licenses.

The framework is technically feasible on modest hardware. Privacy training and attacks ran on a laptop GPU with 4 GB of memory, and detector training ran on a CPU. The main libraries we depend on are free and open source. The main costs would be staff time to adapt the framework to each customer's data and models, plus GPU time for larger detectors. Because the design is file-based, each component can run on a different machine owned by a different team.

Table 7.3 summarizes the main parts of the business model. These are plans, not results. They show how the framework could be sustained after the project ends.

**Table 7.3: Main parts of the business model**

| Element | Plan |
|---|---|
| Customer segments | Autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers |
| Channels | Open-source release of the core, the project website and the research paper |
| Key partners | Open-source communities of the libraries we use, universities and automotive data teams |

| Element | Plan |
|---|---|
| Key resources | The code of the four components, the attack suite, the test suite and the team's knowledge of the pipeline |
| Cost structure | Staff time for integration and support, GPU time for larger models and ongoing maintenance |
| Revenue or sustainability | Paid integration, support and training; a hosted audit service; academic licenses |

Our own results give some evidence of feasibility. On the technical side, the whole pipeline ran on student hardware. Encrypting a 4 KB file took about 15 ms, and each GCM file grows by only 32 bytes. On the operational side, the file-based design worked across four machines owned by four members. It coped with real problems, such as wrong labels and a reporting error, without a redesign.

### 7.4 Adoption intellectual property and compliance

**Adoption barriers.** The utility gap is the biggest barrier. A detector trained on our current synthetic data kept only 10.46% of the real-data mAP50. So developers would still need real data for final training. Before production use, the framework would also need key management, user authentication and an access log.

**Intellectual property.** The framework uses CARLA, PyTorch, Opacus, Ultralytics, PyCryptodome and other open-source libraries. Any release must follow their licenses and keep their copyright notices. We used KITTI only as a utility baseline under its non-commercial license, so a commercial product could not use it in the same way. The rules of the university decide who owns the code written by the group.

**Data protection and ethics.** All images were generated in CARLA, so no real person, vehicle or place was recorded. Files that could reveal source values, such as the calibration audit log, were shared only with the auditing member and never published. The framework does not claim legal certification. It helps a team prepare evidence for duties such as those in GDPR Articles 5, 25 and 32 [3]. The final legal decision stays with the data controller. Any product must state its limits clearly, so that users do not trust the guarantee more than they should.

**Safety.** The framework produces training data and evidence, not a driving system. We did not test any model in a vehicle. A detector trained on our current synthetic data is a

research result. It should not be used for safety decisions without further training and validation on real data.

**Professional responsibilities.** As cybersecurity students, we tried to follow the same duties we expect from others. We reported results that did not meet their target, such as GO3, and we reported the calibration error in full. Keys were removed from the repository once the code review found them. Files that could reveal source values were kept within the team. We cited every library and dataset we used and kept to their licenses.

### 7.5 Sustainable Development Goal contribution

As Section 1.3 described, the likely contribution is to SDG 9 (target 9.5) and SDG 16 (target 16.10). The framework gives the automotive sector a tested way to innovate with synthetic data while protecting the privacy of road users. Its evidence also supports the oversight work of regulators.

The framework could have some negative effects. A signed certificate could give a false sense of security if users read the privacy budget without the attack results and limitations. Training many models uses energy. To reduce these risks, the attack results and limitations should always be published with the certificate. Where a small model is enough, it should be used. A contribution to road safety under SDG 3 and SDG 11 would need the utility gap to be closed first. Only then could synthetic data replace more real data in practice.

Some measures would help keep these benefits over time. The attack suite should be kept up to date as stronger attacks appear, and it should be run again whenever the pipeline changes. The signed certificate should always travel with its attack results and limitations. A future version should log the energy used by each training run. Then the environmental cost could be weighed against the privacy benefit.

# 8 CONCLUSION AND FUTURE DIRECTIONS

## 8.1 Overall conclusion

The common problem had four parts. The pipeline had to apply formal privacy where real information enters and protect its files. It also had to measure its utility and give evidence that an independent party can verify. The integrated evidence answers each of these parts.

Privacy was applied at both entry points. The calibration statistics were privatized before they reached the simulator. DP-SGD brought membership inference down from an AUC of 0.639 to near chance on every private model. The measured cost was 0.16 to 0.21 in test AUC. The files were protected from the moment of capture. The security audit and the real folder scan showed that nobody could read or change them without detection. Utility was measured against real data. The synthetic data trained a working detector within its own domain, but on KITTI images it kept only 10.46% of the real-data mAP50. Every privacy claim was checked by separately written attacks, and all six passed.

So for privacy, protection and verification, the answer is yes. For utility, the answer is not yet. The gap between synthetic and real data is the main problem that remains.

For a developer, this answer has a practical meaning. Our framework can already be used to bring real statistics into a simulator with a formal guarantee, and to keep the generated files safe. It produces evidence that a data protection officer can read and an auditor can test. But the synthetic data cannot yet replace real data for the final training of a detector. A developer would still need real data. The framework would also need captures at several budgets to show how much of the gap comes from privacy.

### 8.2 Principal integrated contribution

The main contribution of our team is a working pipeline that applies, protects, measures and verifies privacy in one flow. Four things came out of the integration that no single component could have produced alone.

- **Evidence that crosses boundaries.** Signed reports, budget logs and manifests let one component check another without trusting it. This is how the composition error in the calibration budget was found.
- **A shared definition of done.** Every claim needs a number, an attack or test result and a signature. After the panel review, we added that every test also needs a positive control.
- **Two levels of privacy cost.** Privacy Integration measured the cost at the model level, and Utility Evaluation measured it at the data level. Together they show where privacy costs the most.
- **Contracts that make the pipeline modular.** File contracts let four components be built on different machines and joined without changing their internal code.

#### 8.2.1 Lessons learned

The project taught us lessons that we would apply to any similar system.

- **Check the data before blaming privacy.** When our first private models failed, it was easy to blame DP-SGD. The real cause was label noise and stretched images. A careful look at the labels saved us from a wrong conclusion.
- **Always attack an unprotected model too.** A result near 50% means nothing unless the same attack works on a model without protection. The non-private baseline made every private result believable.
- **Sign evidence, then let someone else check it.** The calibration total was reported wrongly until an independent check found it. Signed files made that check possible.
- **Agree on contracts early, including model classes.** Most of our integration problems came from details that were missing from a contract. Examples are file names, model checkpoints and column names.

- **Keep security code separate from the simulator.** Because the security classes did not depend on CARLA, they could be tested anywhere. This made the security audit repeatable.
- **Report per class and per level.** Overall scores hid the near-zero results for pedestrians and cyclists. Looking at the cost at both the model level and the data level gave a fairer picture.

### 8.3 Future directions

We list the future work in order of priority.

1. **Close the utility gap.** Capture the dataset at several calibration budgets to draw a full privacy and utility curve. Use a real baseline with a closer image shape, such as Cityscapes. Then fine-tune the synthetic model on a small real subset.
2. **Add positive controls to every test.** Stress-test DP against a strongly overfitted baseline. Plant label changes to test the statistical poisoning check, and plant known test values to test the metadata scan.
3. **Fix the calibration budget.** Run each query at $\varepsilon = 1.8 / 7 \approx 0.257$ so that the total stays near 1.8. Take the source statistics from a real dataset.
4. **Strengthen file protection.** Move the delivered dataset to AES-256-GCM. Encrypt frames in memory so that plaintext is never written to disk. Keep keys in a key management service, with per-batch keys so that crypto-shredding is possible.
5. **Harden access and evidence.** Add user authentication and a tamper-evident access log to the RBAC gateway. Replace HMAC with public-key signatures so that any third party can verify the reports.
6. **Stronger attacks and more seeds.** Use the likelihood ratio attack of Carlini et al. [6] and more seeds per configuration. Bring all compliance reports together in one dashboard for reviewers.

Table 8.1 links each future direction to the components involved and the group objective it would strengthen.

**Table 8.1: Future directions mapped to components and group objectives**

| Priority | Future direction | Components | Objective strengthened |
|---|---|---|---|
| 1 | Close the utility gap | DG, UE | GO3 |
| 2 | Add positive controls to every test | PI, UE, RC | GO2, GO6, GO7 |
| 3 | Fix the calibration budget | PI, DG | GO4 |
| 4 | Strengthen file protection | DG | GO5 |
| 5 | Harden access and evidence | DG, PI, RC | GO5, GO7 |
| 6 | Stronger attacks and more seeds | PI, RC | GO1, GO2 |

The first three items have the most value for the common problem. Closing the utility gap would answer the one part of the problem that is still open. Positive controls would make every existing pass result stronger. Fixing the calibration budget would bring GO4 within its limit. The last three items are about moving from a research pipeline to a system that others could deploy.

# REFERENCES


[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354-3361.

[2] P. Sun *et al.*, "Scalability in perception for autonomous driving: Waymo Open Dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Off. J. Eur. Union*, vol. L 119, pp. 1-88, May 2016.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 3-18.

[6] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 1897-1914.

[7] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1451-1468.

[8] C. Gao, G. Wang, W. Shi, Z. Wang, and Y. Chen, "Autonomous driving security: State of the art and challenges," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7572-7595, 2022.

[9] Y. Xu, J. Wei, T. Mi, and Z. Chen, "Data security in autonomous driving: Multifaceted challenges of technology, law, and social ethics," *World Electr. Veh. J.*, vol. 16, no. 1, Art. no. 6, 2025.

[10] C. Dwork, "Differential privacy," in *Proc. 33rd Int. Colloq. Automata, Lang. Program. (ICALP)*, ser. Lecture Notes Comput. Sci., vol. 4052. Berlin, Germany: Springer, 2006, pp. 1-12.

[11] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211-407, 2014.

[12] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory Cryptogr. Conf.*

*(TCC)*, ser. Lecture Notes Comput. Sci., vol. 3876. Berlin, Germany: Springer, 2006, pp. 265-284.

[13] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 308-318.

[14] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, 2017, pp. 263-275.

[15] A. Yousefpour *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.

[16] Y. Wu and K. He, "Group normalization," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3-19.

[17] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou, "Differentially private generative adversarial network," 2018, *arXiv:1802.06739*.

[18] R. Torkzadehmahani, P. Kairouz, and B. Paten, "DP-CGAN: Differentially private synthetic data and label generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2019, pp. 98-104.

[19] Y. Tian, Y. Shi, Y. Zhang, and Q. Tian, "Personalized federated learning scheme for autonomous driving based on correlated differential privacy," *Sensors*, vol. 25, no. 1, Art. no. 178, 2025.

[20] C. Xie, Z. Cao, Y. Long, D. Yang, D. Zhao, and B. Li, "Privacy of autonomous vehicles: Risks, protection methods, and future directions," 2022, *arXiv:2209.04022*.

[21] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE 31st Comput. Secur. Found. Symp. (CSF)*, 2018, pp. 268-282.

[22] B. Jayaraman and D. Evans, "Evaluating differentially private machine learning in practice," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1895-1912.

[23] M. Nasr, S. Song, A. Thakurta, N. Papernot, and N. Carlini, "Adversary instantiation: Lower bounds for differentially private machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 866-882.

[24] T. Sun *et al.*, "SHIFT: A synthetic driving dataset for continuous multi-task domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 21371-21382.

[25] H. Caesar *et al.*, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11621-11631.

[26] A. Giannaros *et al.*, "Autonomous vehicles: Sophisticated attacks, safety issues, challenges, open topics, blockchain, and future directions," *J. Cybersecur. Privacy*, vol. 3, no. 3, pp. 493-543, 2023.

[27] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[28] *Advanced Encryption Standard (AES)*, FIPS PUB 197, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001, updated 2023.

[29] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Spec. Publ. 800-38D, 2007.

[30] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779-788.

[31] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*. (2023). Ultralytics. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[32] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, ser. Lecture Notes Comput. Sci., vol. 8693. Cham, Switzerland: Springer, 2014, pp. 740-755.

[33] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980-2988.

[34] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 6626-6637.

[35] Z. Wang, J. Ma, X. Wang, J. Hu, Z. Qin, and K. Ren, "Threats to training: A survey of poisoning attacks and defenses on machine learning systems," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1-36, 2022.

[36] J. Hayes, L. Melis, G. Danezis, and E. De Cristofaro, "LOGAN: Membership inference attacks against generative models," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 1, pp. 133-152, 2019.

[37] B. van Breugel, H. Sun, Z. Qian, and M. van der Schaar, "Membership inference attacks against synthetic data through overfitting detection," in *Proc. 26th Int. Conf. Artif. Intell. Statist. (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 206, 2023.

[38] S. K. Murakonda and R. Shokri, "ML Privacy Meter: Aiding regulatory compliance by quantifying the privacy risks of machine learning," 2020, *arXiv:2007.09339*.

[39] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557-570, 2002.

[40] A. Pyrgelis, C. Troncoso, and E. De Cristofaro, "Knock knock, who's there? Membership inference on aggregate location data," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018.

[41] O. Adeboye, T. Dargahi, and M. Babaie, "DeepClean: A robust deep learning technique for autonomous vehicle camera data privacy," *IEEE Access*, vol. 10, 2022, doi: 10.1109/ACCESS.2022.3222834.

[42] K. Parikh, "Driving privacy forward: Mitigating information leakage within smart vehicles through synthetic data generation," 2024, *arXiv:2410.08462*.

[43] Z. Tang *et al.*, "PAMTRI: Pose-aware multi-task learning for vehicle re-identification using highly randomized synthetic data," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 211-220.

[44] *Information Security, Cybersecurity and Privacy Protection: Privacy Enhancing Data De-identification Framework*, ISO/IEC 27559:2022, Int. Org. Standardization, Geneva, Switzerland, 2022.

[45] G. Ganev, "When synthetic data met regulation," 2023, *arXiv:2307.00359*.

[46] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[47] Nat. Inst. Standards Technol., "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," U.S. Dept. Commerce, Gaithersburg, MD, USA, Rep. NIST AI 100-1, 2023.

[48] PyCryptodome Project, "PyCryptodome documentation," 2024. Accessed: Oct. 10, 2026. [Online]. Available: https://www.pycryptodome.org

[49] ReportLab, "ReportLab PDF library user guide," 2024. Accessed: Oct. 10, 2026. [Online]. Available: https://www.reportlab.com/docs/reportlab-userguide.pdf

# APPENDICES

## Appendix A Integrated interface specification

This appendix describes the files that connect the four components. Before a consumer uses any file in Table A.1, the file is checked against a JSON Schema contract or a fixed format.

**Table A.1: Integrated interface specification**

| Artifact | Producer | Used by | Format | Validation rule |
|---|---|---|---|---|
| calibration_stats.json | PI | DG, RC | JSON: seven privatized statistics, ε per query, total ε | Schema check |
| calibration_audit_log.json | PI | RC only | JSON: noise drawn for each query | Never published; used for reconstruction and composition checks |
| Encrypted image files | DG | UE, PI, RC | 16-byte IV followed by AES-256-CBC ciphertext with PKCS7 padding | Decrypts with the dataset key |
| Town manifests | DG | UE, PI, RC | SHA-256 digest of each original file | Digest of each decrypted file must match |
| Label files | CARLA capture | UE | YOLO format: class, normalized x, y, width, height | Structural and statistical poisoning gate |
| clean_labels/ | UE | PI | Label files that passed the gate | PASS verdict required before training |
| audit_report.json and audit_certificate.json | PI | RC | JSON report with ε, C, σ, δ, limit and steps; HMAC-SHA256 signature and SHA-256 digest | Signature and digest must match |
| Budget logs | PI | RC | Cumulative ε after every training step | Final value within 0.0001 of report; never decreasing; below 10 |

| Artifact | Producer | Used by | Format | Validation rule |
|---|---|---|---|---|
| summary.json | PI | UE | Attack AUC and test AUC per configuration | Schema check |
| Attack reports | RC | Release folder | JSON with a shared schema, plus a PDF | Combined by the master script |

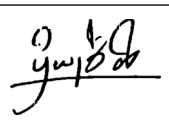
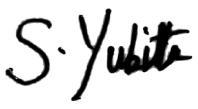
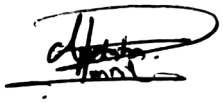
## Appendix B Team contribution declaration

All four members agreed on the contributions in Table B.1. Each member owned one component, plus their share of the integration work described in Section 4.5.

**Table B.1: Team contribution declaration**

| Student ID | Student name | Individual component | Contribution |
|---|---|---|---|
| IT22309556 | Priyadarshi S.A.S.D. | Privacy Integration with Differential Privacy | 25% |
| IT22541284 | Yubitha S. | Secure Synthetic Dataset Generation | 25% |
| IT22110220 | Apeksha M.K.S.R | Utility Evaluation of Synthetic Data | 25% |
| IT22066916 | Wijesinghe R.D.P.T. | Risk and Compliance Auditing | 25% |

Confirmed by all four members:

| Student name | Student ID | Signature |
|---|---|---|
| Priyadarshi S.A.S.D. | IT22309556 |  |
| Yubitha S. | IT22541284 |  |
| Apeksha M.K.S.R | IT22110220 |  |
| Wijesinghe R.D.P.T. | IT22066916 |  |

## Appendix C Academic integrity and AI use disclosure

We used generative AI while preparing this report, following the module policy on AI use.

**Tool.** Claude, a large language model by Anthropic.

**Use.** The tool was used to create the figures and diagrams from our results, and to help with small verification checks on the document.

**Verification.** We checked the numbers, citations and claims in the report against the individual reports, the experiment outputs and the cited sources.

**Responsibility.** The research design, the experiments, the results and the conclusions are the responsibility of the team.